GOVERNMENT COLLEGE OF ENGINEERING
DHARMAPURI – 636 704

(Affiliated to Anna University, Chennai)



BONAFIDE CERTIFICATE

NAME :.....

REG.NO :.....

DEGREE & BRANCH : B.E&ECE.....

SUBJECT CODE&TITLE: NM1133& EmbeddedC&MicroController Programming

YEAR & SEM : IV&VII.....

Certified that this Bonafide Record of Work done by the above student of the

..... *Laboratory*

during the year 20 - 20 .

Faculty In-Charge **Head of the Department**

Submitted for the University Practical Examination held on.....at

Government College of Engineering, Dharmapuri.

Internal Examiner **External Examiner**

CONTENTS

Exp. No.	Date	Name of Experiment	Page No.	Marks	Signature
1.		Develop a Real-Time Soil Monitoring and Automated Irrigation Control System	6		
2.		Develop a Smart Home Automation for Lighting, Temperature, and Appliances	9		
3.		Develop a Industrial Machine Health Monitoring and Predictive Maintenance System	12		
4.		Develop a Wearable Vital-Sign and Sleep-Quality Monitoring System	15		
5.		Develop a Smart City Traffic, Waste, and Pollution Monitoring system	18		
6.		Develop a Environmental Monitoring System: Pollution, Wildfire, and Wildlife Observation	21		
7.		Develop a Sensor-Based Failure Detection and Maintenance Scheduling system	24		
8.		Develop a Smart Retail Inventory and Customer-Experience Monitoring	27		
9.		Develop a Smart Logistics: Live Tracking and Shipment Security	30		
10.		Develop a Connected Vehicle Performance and Remote Diagnostics System	33		

Exp. No.	Date	Name of Experiment	Page No.	Marks	Signature
11.		Develop a Smart Building Facility Management and Access Control system	36		
12.		Develop a Smart Grid Monitoring and Renewable-Energy Integration system	39		
13.		Develop a Precision Agriculture system: Resource Use, Crop Health, and Yield Support	42		
14.		Develop a Smart Livestock Monitoring, Location Tracking, and Theft Detection system	45		
15.		Develop a Remote Property Monitoring and Smart Security system	48		
16.		Develop a Smart Water Quality Monitoring system	51		
17.		Develop a Smart Parking Management and Payment-System Demonstrator	54		
18.		Develop a Construction-Site Worker Safety and Operations Monitoring System	57		
19.		Develop a Smart Waste Management and Sustainable Recycling System	60		

List of Experiments

1) Develop a Real-Time Soil Monitoring and Automated Irrigation Control System.
2) Develop a Smart Grid Monitoring and Renewable-Energy Integration system.
3) Develop a Industrial Machine Health Monitoring and Predictive Maintenance System.
4) Develop a Wearable Vital-Sign and Sleep-Quality Monitoring System.
5) Develop a Smart City Traffic, Waste, and Pollution Monitoring system.
6) Environmental Monitoring System: Pollution, Wildfire, and Wildlife Observation.
7) Develop a Sensor-Based Failure Detection and Maintenance Scheduling system.
8) Develop a Smart Retail Inventory and Customer-Experience Monitoring.
9) Develop a Smart Logistics: Live Tracking and Shipment Security.
10) Develop a Connected Vehicle Performance and Remote Diagnostics System.
11) Develop a Smart Building Facility Management and Access Control system.
12) Develop a Smart Grid Monitoring and Renewable-Energy Integration system.
13) Develop a Precision Agriculture system: Resource Use, Crop Health, and Yield Support.
14) Develop a Smart Livestock Monitoring, Location Tracking, and Theft Detection system.
15) Develop a Remote Property Monitoring and Smart Security system.
16) Develop a Smart Water Quality Monitoring system.
17) Develop a Smart Parking Management and Payment-System Demonstrator.
18) Develop a Construction-Site Worker Safety and Operations Monitoring System.
19) Develop a Smart Waste Management and Sustainable Recycling System.

1. Introduction and Design Method

Embedded C and Microcontroller Programming is the process of developing software to control hardware devices using microcontrollers. Embedded C is a programming language commonly used to write programs that interact with input devices, sensors, actuators, displays, and communication modules. A typical embedded system consists of a microcontroller, input devices, output devices, a power supply and a software program that processes data and controls the connected hardware.

The design method involves identifying the system requirements, selecting a suitable microcontroller, designing the circuit connections, writing and compiling the Embedded C program, uploading the program to the controller, and testing the system through simulation or hardware implementation. The microcontroller reads input signals, processes them according to the programmed instructions, and generates the required output. This approach is widely used in automation, robotics, IoT applications, industrial control, and smart electronic devices.

1.1 Arduino Uno and ESP32

Feature	Arduino Uno (typical R3)	ESP32 development board (typical)
Logic / GPIO	5 V logic on classic ATmega328P Uno	3.3 V logic; GPIO is generally not 5 V tolerant
Analog input	10-bit ADC; A0–A5 available	ADC capability/pins vary by chip and board; many use 12-bit readings
Connectivity	No built-in Wi-Fi/Bluetooth on classic Uno	Wi-Fi and Bluetooth available on many ESP32 variants
Typical use	Simple local sensing and actuation	Connected nodes, more complex sensing, remote dashboards
Important check	Use correct board pin labels and library versions	ADC range, pin strapping, USB/serial pin conflicts vary by board

1.2 General workflow

1. Define the physical quantity and intended decision.
2. Select a sensor with a suitable range, interface, and accuracy.
3. Check supply voltage, signal levels, current, grounding, and environmental protection.
4. Wire and test the sensor alone before adding actuators.
5. Calibrate with known reference conditions or values.
6. Implement filtering, threshold logic, invalid-data handling, and logging.
7. Test normal, boundary, fault, and recovery conditions.
8. Record measured results, uncertainty, and deviations from expected behavior.

1.3 General wiring and code cautions

- Never drive motors, pumps, relays, solenoids, or buzzers beyond GPIO ratings; use a suitable driver.
- A relay module may be active-LOW or active-HIGH. Confirm its behavior before connecting a load.
- For ESP32 boards, do not feed 5 V into GPIO or ADC pins unless the board documentation explicitly supports it.

EXP NO :1 Real-Time Soil Monitoring and Automated Irrigation Control DATE:

Application domain: Smart agriculture

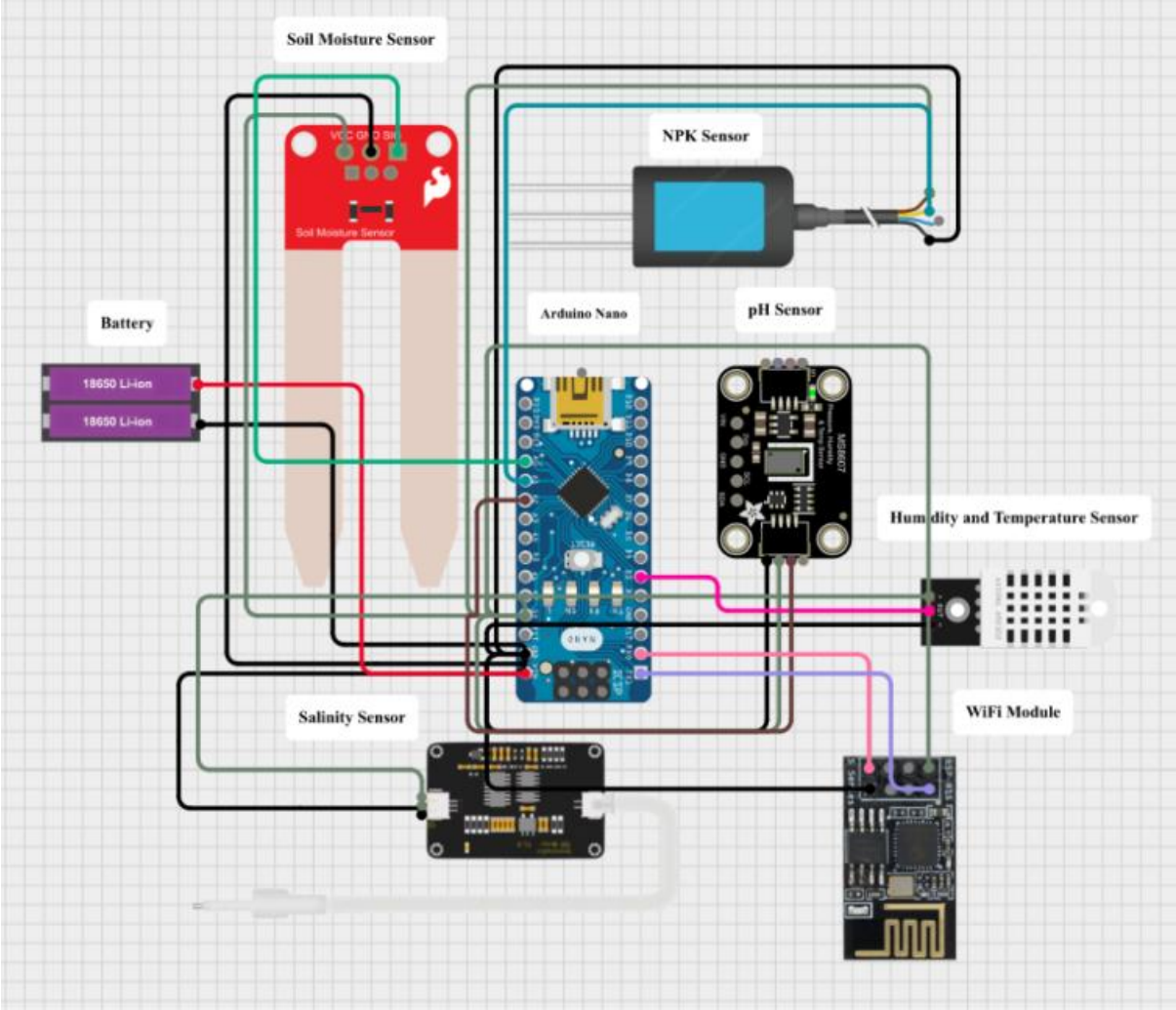


Figure 1 : Smart Agriculture: illustrative system block diagram and project visual.

Abstract

Measures soil moisture and optionally temperature, then switches a water pump when the soil is dry. The design separates low-voltage sensing/control from the pump power circuit.

Objectives

- Measure soil moisture at regular intervals.
- Set an adjustable moisture threshold for irrigation.
- Prevent rapid relay switching with hysteresis and a minimum pump run interval.
- Provide a clear status output for dry soil, watering, and adequate moisture.

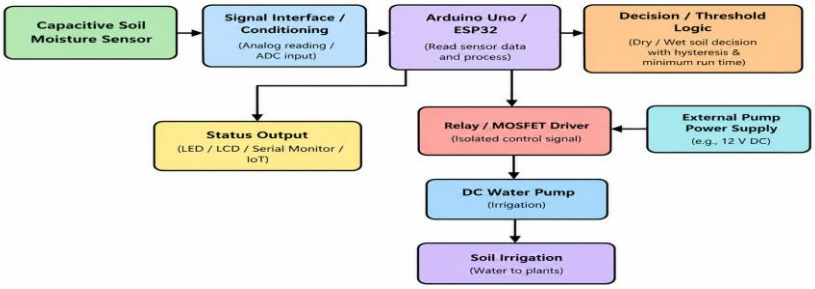
Components Required

Arduino Uno or ESP32; capacitive soil-moisture sensor; 5 V relay module or logic-level MOSFET driver; suitable DC pump; external pump supply; flyback protection if using a bare inductive driver; jumper wires; optional DHT22 and LCD.

Interface and Pin Connections

Moisture sensor AO → A0 (Uno) or an ADC-capable ESP32 pin such as GPIO34; sensor VCC → voltage supported by its module; sensor GND → controller GND. Driver input → D7 (Uno) or GPIO26 (ESP32). Pump uses a separate supply through a properly rated driver/relay; never power the pump from a GPIO pin.

Block Diagram



Working Principle

Capacitive probes estimate water content from changes in dielectric properties and are generally more corrosion-resistant than exposed resistive probes. Calibrate readings in dry and wet soil. A threshold plus hysteresis avoids relay chatter.

Implementation Procedure

Record the sensor reading in dry soil and in well-watered soil; choose a threshold from these measurements.

1. Connect the sensor and driver with the pump supply disconnected.
2. Upload the code and verify status messages using Serial Monitor.
3. Test the driver with a low-voltage lamp or meter before connecting the pump.
4. Check for leaks, dry-run risk, and safe electrical separation before field use.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// Arduino Uno example: capacitive sensor + relay module
const int MOISTURE_PIN = A0;
const int RELAY_PIN = 7;
const int DRY_THRESHOLD = 650; // calibrate for your sensor
const int WET_THRESHOLD = 580; // hysteresis; calibrate
bool watering = false;

void setup() {
  Serial.begin(9600);
  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, HIGH); // assumes active-LOW relay; verify module
}
void loop() {
```

```
  int raw = analogRead(MOISTURE_PIN);
  Serial.print("Moisture raw: "); Serial.println(raw);
  if (!watering && raw >= DRY_THRESHOLD) watering = true;
  if (watering && raw <= WET_THRESHOLD) watering = false;
  digitalWrite(RELAY_PIN, watering ? LOW : HIGH);
  Serial.println(watering ? "IRRIGATION REQUESTED" : "SOIL OK");
  delay(1000);
      .
}
```

Proposed Test Plan

Test condition	Expected observation
Dry soil	Reading crosses the dry threshold; irrigation output is requested.
Moist soil	Pump output remains off.
Sensor disconnected	Reading is treated as invalid where possible; system should fail safely.
Repeated threshold crossing	Hysteresis/minimum run timing prevents rapid switching.

Expected Results

In bench tests, readings should change between dry and wet soil and the controller should switch the output at the chosen threshold. These are expected outcomes, not experimentally verified results.

Limitations, Safety, and Future Scope

Sensor readings depend on soil type, salinity, probe placement, and supply voltage. Use a pump driver rated for the motor's startup current, add a water-level interlock, and avoid leaving a pump unattended.

Future scope: Add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.

EXP NO:2 Smart Home Automation for Lighting, Temperature, and Appliances

DATE:

Application domain: Smart buildings

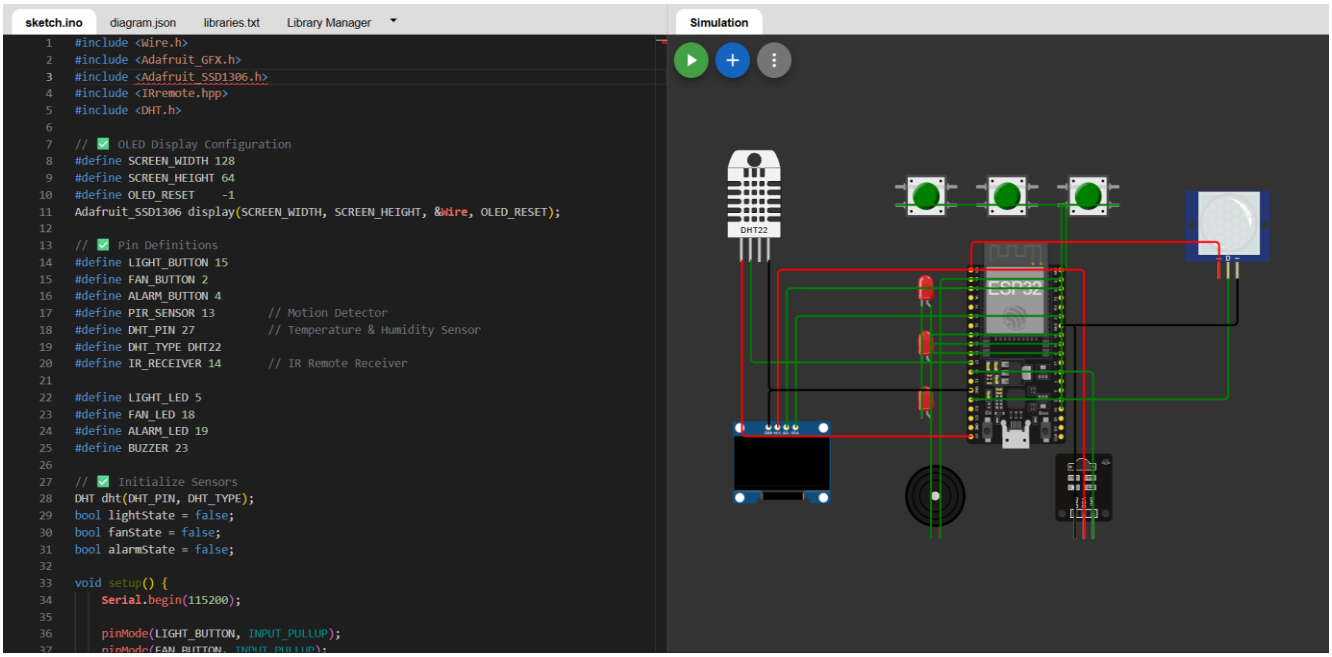


Figure 2. Smart Home Automation for Lighting, Temperature, and Appliances

Abstract

Combines occupancy/light and temperature sensing with controlled lighting and a low-voltage appliance demonstrator. Real household mains loads require an enclosed, certified switching device and qualified installation.

Objectives

- Measure ambient light and room temperature.
- Switch a demonstration light based on light level or occupancy.
- Apply a temperature threshold to a fan demonstrator.
- Expose configurable thresholds for routine-based automation.

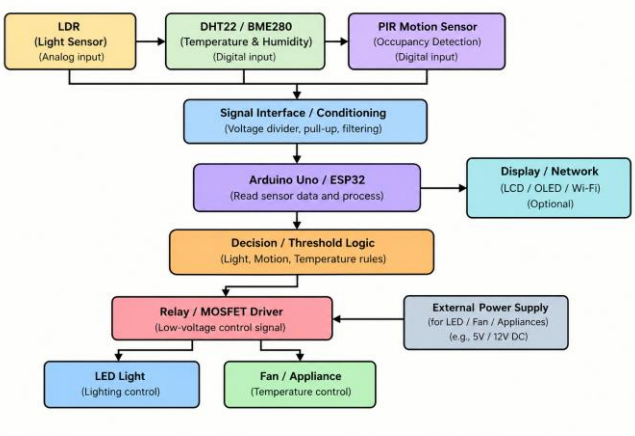
Components Required

Arduino Uno or ESP32; LDR with 10 kΩ resistor or BH1750; DHT22/BME280; PIR motion sensor; relay module or MOSFET driver; LED/fan demonstrator; display or Wi-Fi dashboard optional.

Interface and Pin Connections

LDR divider midpoint → A0 (Uno) or ADC pin (ESP32); DHT22 data → D2 (Uno) or GPIO4 (ESP32), with pull-up if required; PIR OUT → D3/GPIO27; low-voltage driver input → D8/GPIO26. Share controller ground for low-voltage modules.

Block Diagram



Working Principle

The LDR voltage divider converts light level into an analog value. A temperature sensor provides environmental data, while a PIR detects changes in infrared energy. Rules combine these signals to make automation decisions.

Implementation Procedure

1.Calibrate the LDR value in bright and dim conditions.
2.Read temperature and confirm plausible values in Serial Monitor.
3.Test PIR detection without connecting any mains appliance.
4.Set thresholds and verify the rule logic for occupied/unoccupied conditions.
5.For real mains loads, use a certified enclosed smart switch and follow local electrical codes.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// Arduino Uno: LDR + DHT22 + PIR + low-voltage outputs
#include <DHT.h>
#define DHTPIN 2
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);
const int LDR_PIN=A0, PIR_PIN=3, LIGHT_OUT=8, FAN_OUT=9;
const int DARK_LEVEL=600; // calibrate
const float TEMP_LIMIT=30.0;

void setup() {
  Serial.begin(9600); dht.begin();
  pinMode(PIR_PIN, INPUT);
  pinMode(LIGHT_OUT, OUTPUT); pinMode(FAN_OUT, OUTPUT);
}
void loop() {
  int light = analogRead(LDR_PIN);
  int motion = digitalRead(PIR_PIN);
  float temp = dht.readTemperature();
  bool valid = !isnan(temp);
  digitalWrite(LIGHT_OUT, (light > DARK_LEVEL && motion) ? HIGH : LOW);
```

```
  digitalWrite(FAN_OUT, (valid && temp >= TEMP_LIMIT) ? HIGH : LOW);
  Serial.print("LDR="); Serial.print(light);
  Serial.print(" motion="); Serial.print(motion);
  Serial.print(" tempC="); Serial.println(valid ? temp : -999);
  delay(1000);

}
```

Proposed Test Plan

Test condition	Expected observation
Bright room	Light output follows the configured bright-light rule.
Dark + motion	Demonstration light turns on when the rule conditions .
Temperature above threshold	Fan output is requested.
No motion timeout	Lighting returns to its configured idle state.

Expected Results

The system should react to light, motion, and temperature thresholds once sensors are calibrated. No household energy savings are claimed until measured over a defined period.

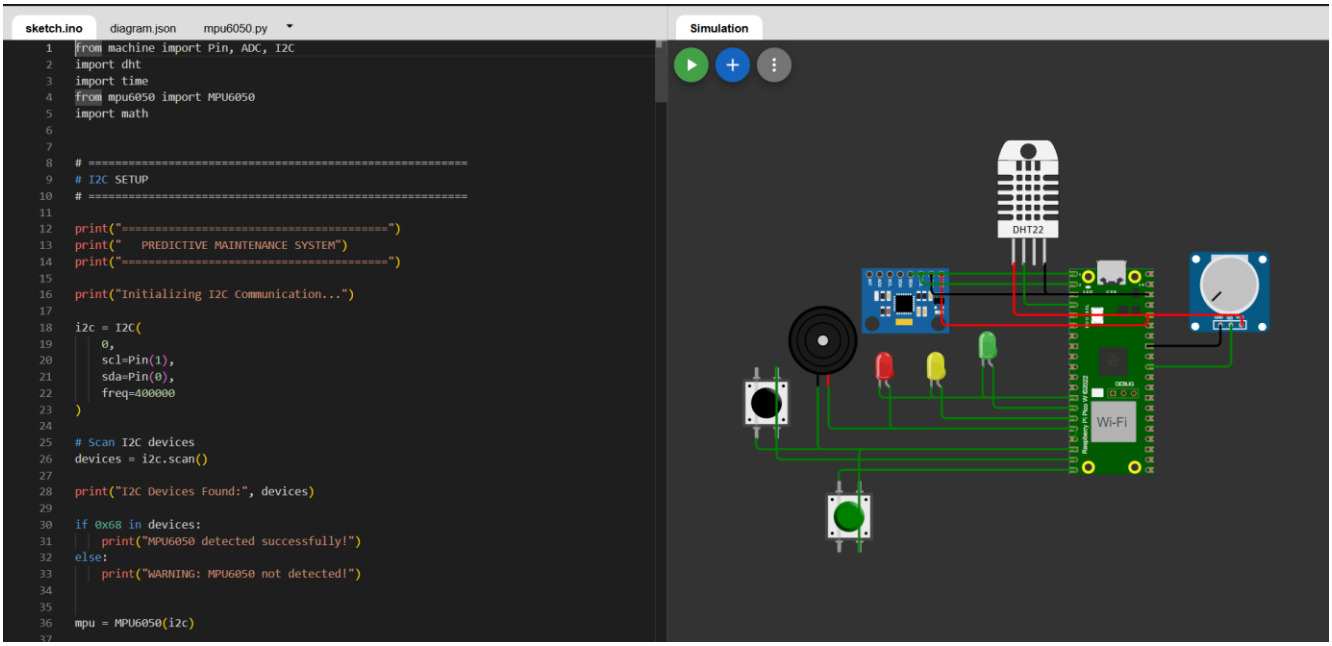
Limitations, Safety, and Future Scope

PIR sensors do not reliably detect a stationary person. Sensor placement, HVAC airflow, and LDR orientation affect results. Never expose mains wiring on a breadboard.

Future scope: Add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.

EXP NO:3 Industrial Machine Health Monitoring and Predictive Maintenance

DATE:




```
from machine import Pin, ADC, I2C
import dht
import time
from mpu6050 import MPU6050
import math


# ========================================================
# I2C SETUP
# ========================================================

print("=====================================")
print("   PREDICTIVE MAINTENANCE SYSTEM")
print("=====================================")

print("Initializing I2C Communication...")

i2c = I2C(
    0,
    scl=Pin(1),
    sda=Pin(0),
    freq=400000
)

# Scan I2C devices
devices = i2c.scan()

print("I2C Devices Found:", devices)

if 0x68 in devices:
    print("MPU6050 detected successfully!")
else:
    print("WARNING: MPU6050 not detected!")


mpu = MPU6050(i2c)
```

Figure 3. Industrial Machine Health Monitoring: illustrative system block diagram and project visual.

Application domain: Industrial IoT

Abstract

Demonstrates condition monitoring by tracking vibration and temperature. It records threshold events that can indicate imbalance, looseness, overheating, or changing operating conditions.

Objectives

- Acquire vibration and temperature measurements.
- Establish a baseline during normal operation.
- Flag measurements outside configured limits.
- Demonstrate a maintenance alert without directly controlling industrial machinery.

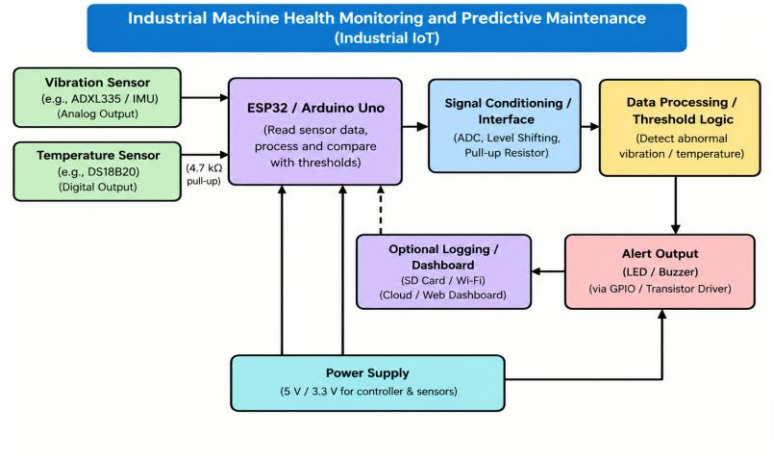
Components Required

ESP32 or Arduino Uno; vibration sensor (analog accelerometer such as ADXL335 or digital IMU); DS18B20/thermistor; status LED/buzzer; optional SD card or Wi-Fi dashboard.

Interface and Pin Connections

Analog vibration output → ADC pin; DS18B20 data → GPIO4/D2 with 4.7 kΩ pull-up to the sensor logic supply; LED → resistor → GPIO; buzzer through a transistor driver if current exceeds GPIO limits.

Block Diagram



Working Principle

Vibration amplitude and temperature trends can reveal changes in machine condition. A simple threshold is suitable for a teaching demonstrator; robust predictive maintenance requires sampling, frequency analysis, historical data, and machine-specific baselines.

Implementation Procedure

1.Mount the sensor firmly to a safe, non-rotating surface.
2.Collect a baseline under normal operation; do not touch moving machinery.
3.Set alert thresholds from machine documentation and baseline data.
4.Run the monitor and log values at consistent intervals.
5.Compare trend records and validate any alert with an appropriate inspection.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// ESP32 example: analog vibration + DS18B20
#include <OneWire.h>
#include <DallasTemperature.h>
const int VIB_PIN=34, LED_PIN=2;
OneWire oneWire(4);
DallasTemperature tempBus(&oneWire);
const int VIB_LIMIT=2200; // determine from baseline

void setup() {
  Serial.begin(115200); pinMode(LED_PIN, OUTPUT);
  tempBus.begin();
}
void loop() {
  int vib=analogRead(VIB_PIN);
  tempBus.requestTemperatures();
  float t=tempBus.getTempCByIndex(0);
  bool alert=(vib>VIB_LIMIT || t>65.0);
  digitalWrite(LED_PIN, alert ? HIGH : LOW);
  Serial.printf("vibration=%d temperature=%.2f C alert=%s\n",
```

```
  vib,t,alert?"YES":"no");
  delay(500);
}
```

Proposed Test Plan

Test condition	Expected observation
Normal idle	Readings remain near the established baseline.
Controlled vibration stimulus	Vibration indicator responds to a safe tap or test fixture.
Warm sensor stimulus	Temperature rises gradually when safely warmed.
Threshold event	Alert activates and is recorded.

Expected Results

Sensor values should be displayed and configured threshold crossings should trigger an alert. Predictive accuracy is not established by a single threshold demo.

Limitations, Safety, and Future Scope

Do not attach sensors to exposed rotating parts. Thresholds vary by machine, sensor mounting, sampling rate, and vibration frequency. This prototype is not a safety-rated shutdown system.

Future scope: Add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.

EXP NO:4 Wearable Vital-Sign and Sleep-Quality Monitoring
DATE:

Application domain: Wearable health technology

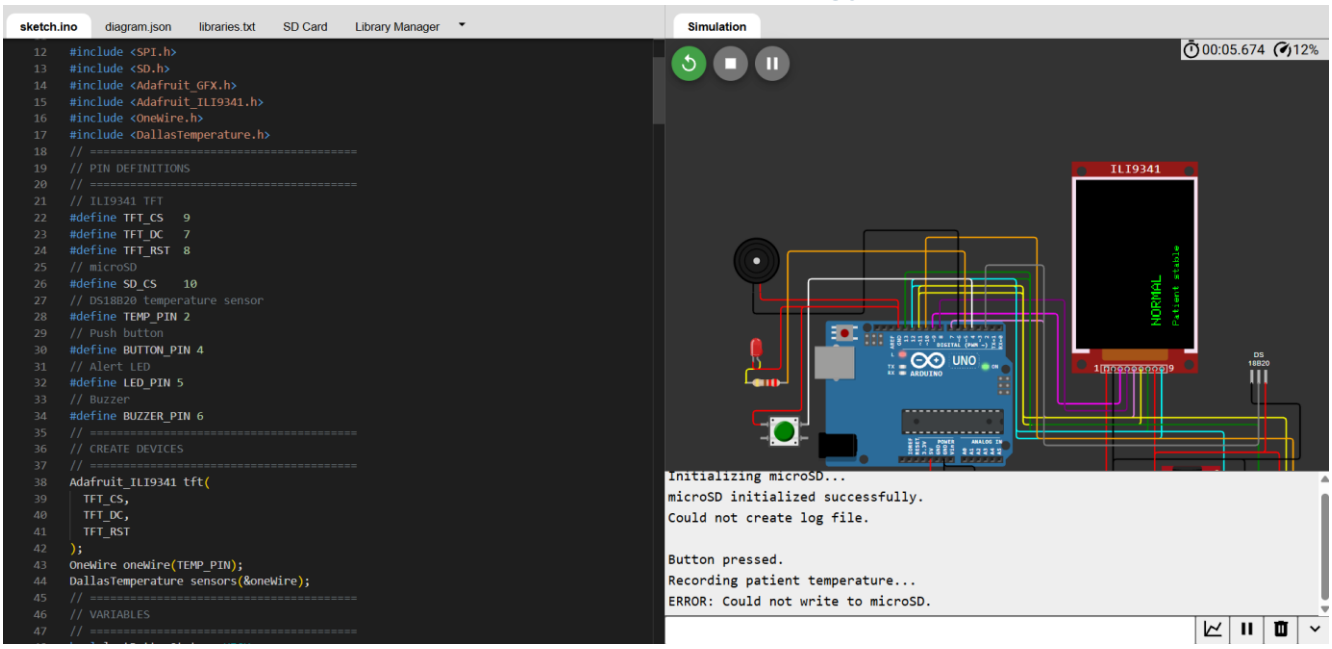


Figure 4:Wearable vital-sign and sleep-quality monitoring

Abstract

A learning prototype records pulse-related optical readings and movement to explore activity and sleep patterns. It is not a medical device and must not diagnose or rule out health conditions.

Objectives

- Read a pulse-oximeter sensor module or heart-rate sensor.
- Track movement using an accelerometer.
- Log time-stamped readings for later review.
- Explain the difference between wellness data and clinical diagnosis.

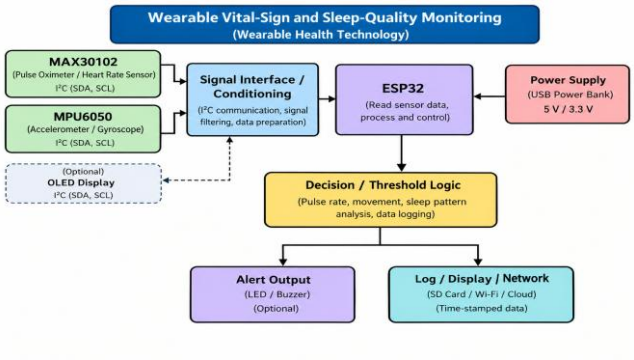
Components Required

ESP32; MAX30102 module; MPU6050 accelerometer/gyroscope; OLED optional; USB power bank; enclosure and strap. Sensor compatibility and libraries must be checked for the exact module.

Interface and Pin Connections

MAX30102 and MPU6050 commonly use I²C: SDA/SCL to the board's I²C pins (Uno A4/A5; ESP32 commonly GPIO21/22). Use 3.3 V-compatible modules and confirm breakout-board voltage support.

Block Diagram



Working Principle

Optical PPG sensors estimate pulse-related changes in blood-volume absorption; accelerometers measure motion. Sleep-stage determination requires validated algorithms and usually more signals than movement alone.

Implementation Procedure

1.Confirm I²C addresses and read sensor samples with a known library example.
2.Keep the optical sensor stable against the skin and avoid excessive pressure.
3.Record pulse-related data and motion alongside timestamps.
4.Compare activity patterns over a night without labeling them as clinical sleep stages.
5.Protect personal data and obtain consent before collecting anyone else's readings.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// ESP32 I2C bring-up template for MAX30102 + MPU6050
// Install SparkFun MAX3010x and Adafruit MPU6050 libraries.
// This template checks the I2C bus; add library-specific acquisition
// after confirming the exact breakout and library versions.
#include <Wire.h>
const int SDA_PIN=21, SCL_PIN=22;
void setup() {
  Serial.begin(115200);
  Wire.begin(SDA_PIN,SCL_PIN);
  Serial.println("I2C scan starting");
  for (byte a=1; a<127; a++) {
    Wire.beginTransmission(a);
    if (Wire.endTransmission()==0) {
      Serial.print("I2C device at 0x");
      Serial.println(a,HEX);
    }
  }
}
void loop() { delay(1000); }
```

Proposed Test Plan

Test condition	Expected observation
Sensor absent	Software reports a sensor error rather than a plausible reading.
Still hand	Motion readings are low and stable.
Gentle movement	Accelerometer values change.
Long recording	Data includes timestamps and missing samples are identified.

Expected Results

The prototype should acquire sensor signals when fit and lighting conditions are suitable. Pulse accuracy and sleep quality must be validated against an appropriate reference before any performance claim.

Limitations, Safety, and Future Scope

Motion, skin tone, perfusion, fit, ambient light, and sensor quality affect readings. Do not use this device for emergency decisions or medical advice.

Future scope: Add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.

EXP NO:5 Smart City Traffic, Waste, and Pollution Monitoring
DATE:

Application domain: Smart city

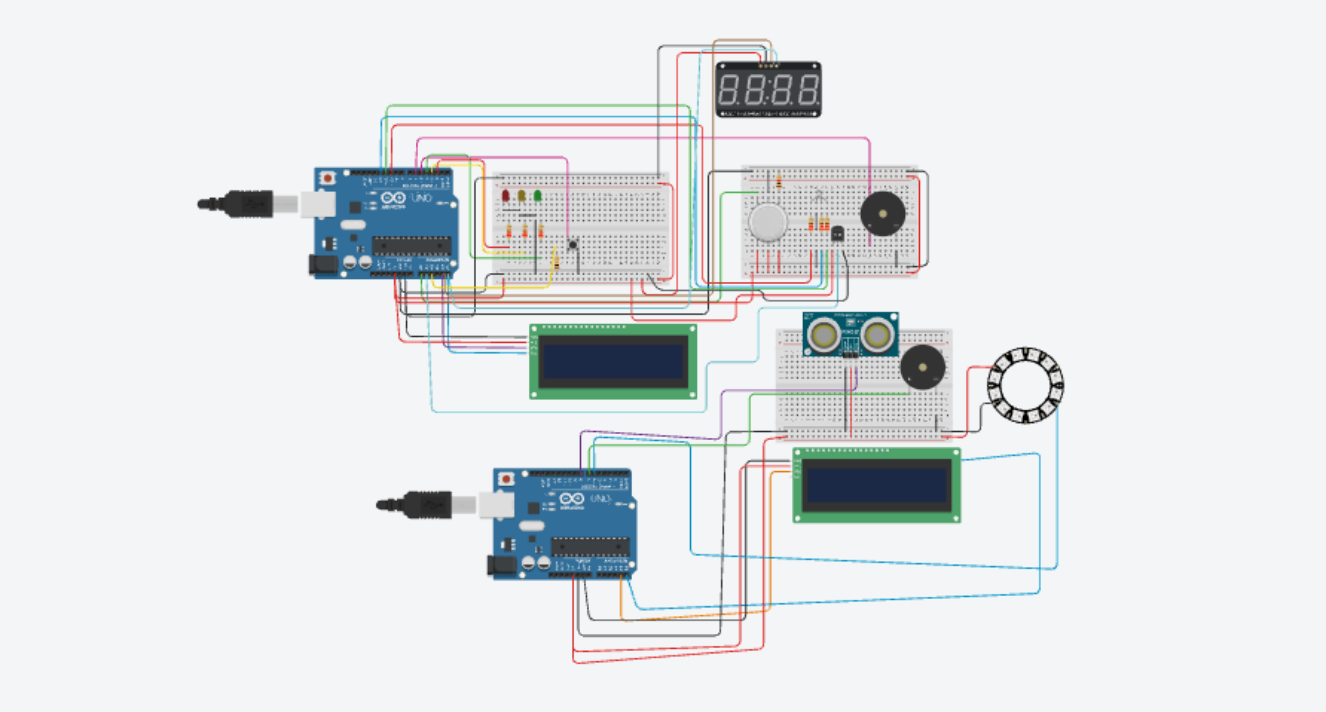


Figure 5:Smart city traffic,waste,and pollution monitoring

Abstract

Combines a traffic counter, fill-level sensor, and environmental sensor readings into a small monitoring node. Each subsystem can be deployed independently before being integrated.

Objectives

- Count vehicle-like events using a break-beam or IR sensor.
- Estimate bin fill level using an ultrasonic sensor.
- Collect air-quality proxy readings from a supported gas sensor.
- Present threshold alerts for monitoring and planning.

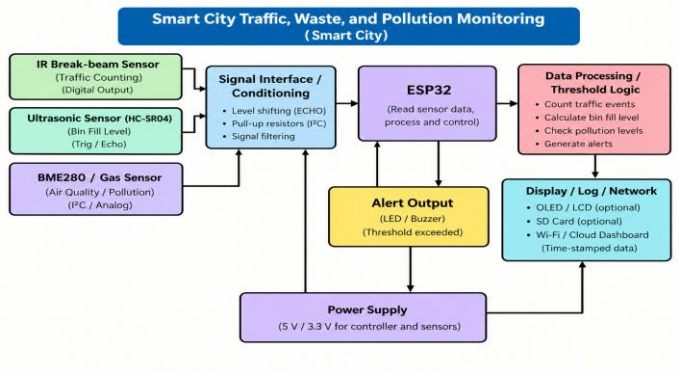
Components Required

ESP32; IR break-beam or reflective sensor; HC-SR04 ultrasonic sensor; BME280 and/or calibrated gas sensor module; LEDs; optional Wi-Fi dashboard. Gas sensors require warm-up and calibration.

Interface and Pin Connections

IR digital OUT → GPIO27; ultrasonic TRIG/ECHO → GPIO5/GPIO18 (use a divider/level shifter on ECHO for ESP32); I²C sensors → GPIO21/22. Never apply 5 V logic directly to ESP32 GPIO.

Block Diagram



Working Principle

Event counting approximates traffic flow at a single point; ultrasonic distance estimates remaining bin space; gas sensors provide indicative readings rather than regulatory air-quality measurements.

Implementation Procedure

1.Test each sensor separately and document its measurement range.
2.Measure the empty-bin and full-bin distances and set limits.
3.Debounce traffic pulses to avoid counting one vehicle multiple times.
4.Publish timestamped data and clearly label sensor estimates.
5.Compare pollution readings with a calibrated reference if they will inform decisions.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// ESP32: ultrasonic bin-level estimate + IR event count
const int TRIG=5, ECHO=18, IR_PIN=27, LED=2;
volatile unsigned long events=0;
unsigned long lastCount=0;
void IRAM_ATTR onEvent() {
  unsigned long now=millis();
  if (now-lastCount>250) { events++; lastCount=now; }
}
float distanceCm() {
  digitalWrite(TRIG,LOW); delayMicroseconds(2);
  digitalWrite(TRIG,HIGH); delayMicroseconds(10);
  digitalWrite(TRIG,LOW);
  long us=pulseIn(ECHO,HIGH,30000);
  if (!us) return -1;
  return us*0.0343/2.0;
}
void setup() {
  Serial.begin(115200);
  pinMode(TRIG,OUTPUT); pinMode(ECHO,INPUT); pinMode(LED,OUTPUT);
  pinMode(IR_PIN,INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(IR_PIN),onEvent,FALLING);
}
void loop() {
```

```
  float d=distanceCm();
  Serial.printf("events=%lu bin_distance_cm=%.1f\n",events,d);
  digitalWrite(LED,(d>0 && d<10)?HIGH:LOW); // near-full alert example
  delay(1000);
}
```

Proposed Test Plan

Test condition	Expected observation
IR beam interruption	Counter increments once per debounced event.
Empty bin	Distance corresponds to a low fill estimate.
Obstacle near sensor	Distance decreases and fill estimate increases.
Gas sensor warm-up	Readings stabilize according to the module datasheet.

Expected Results

The demonstrator should produce traffic-event counts, bin-distance readings, and environmental sensor values. City-scale accuracy is not established by this prototype.

Limitations, Safety, and Future Scope

Traffic events are not necessarily unique vehicles; ultrasonic readings are affected by angled or soft surfaces. Low-cost gas sensors are cross-sensitive and should not be presented as certified pollution measurements.

Future scope: Add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.

EXP NO:6 Environmental Monitoring: Pollution, Wildfire, and Wildlife Observation

DATE:

Application domain: Environmental sensing

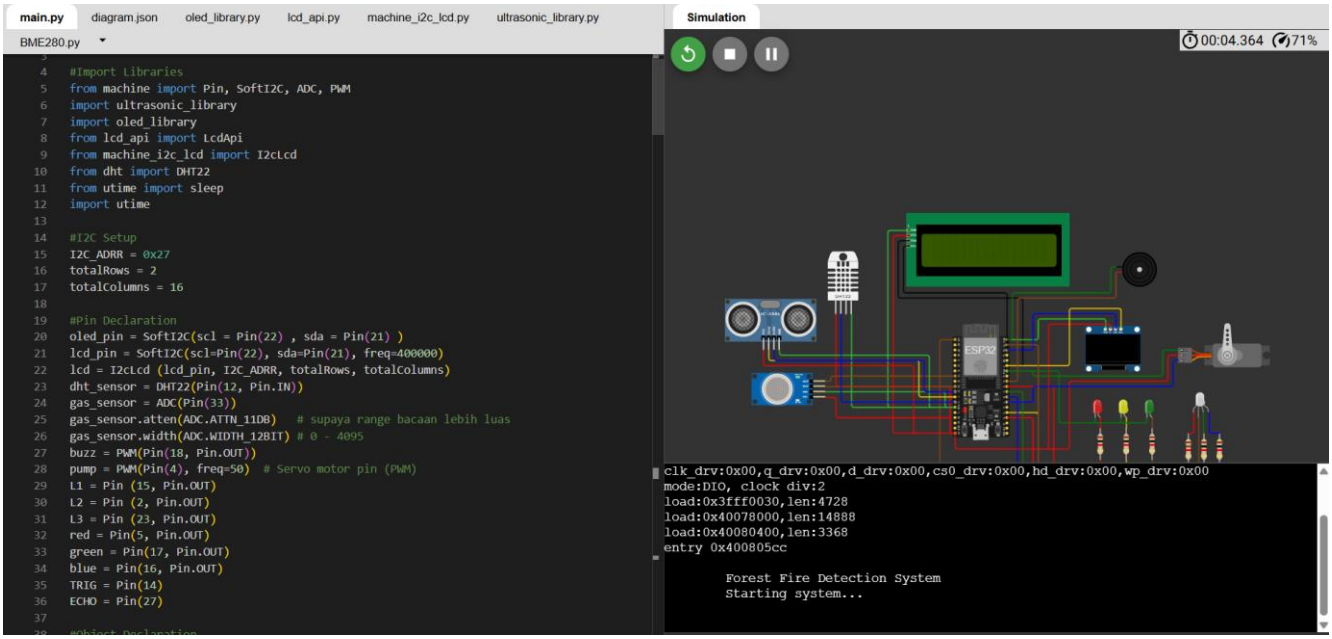


Figure 6:Environmental monitoring:Pollution,Wildfire,And Wildlife Observation

Abstract

Uses temperature, humidity, smoke/gas proxy, and motion sensing to demonstrate environmental event detection. The design is a learning prototype, not a certified wildfire alarm or wildlife-survey instrument.

Objectives

- Record temperature and humidity.
- Detect a configurable rapid temperature rise or sensor threshold.
- Use a PIR or camera-trigger input to record wildlife-like motion events.
- Provide local status and timestamped logging.

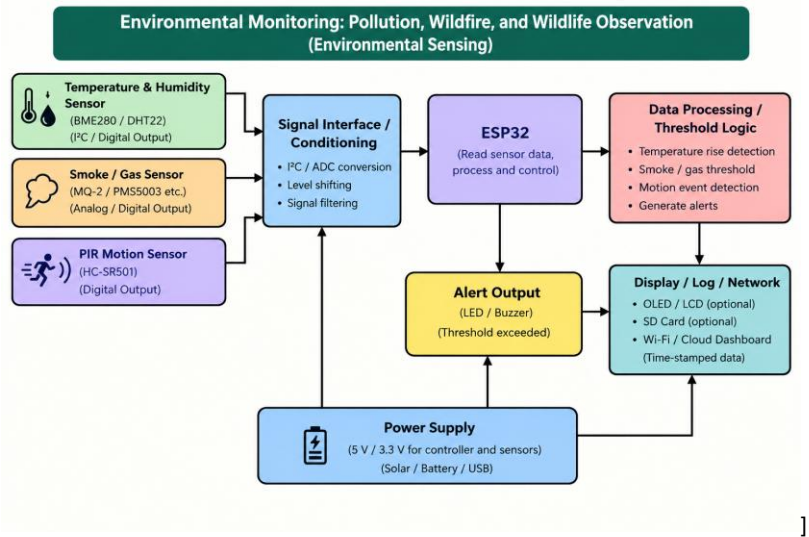
Components Required

ESP32; BME280/DHT22; appropriate smoke/particulate sensor module; PIR or camera trigger; buzzer/LED; weather-resistant enclosure; optional SD card/solar supply.

Interface and Pin Connections

Environmental sensor on I²C or a digital data pin; PIR OUT → GPIO27; status LED → GPIO2 through resistor; buzzer via transistor driver. Keep outdoor electronics dry and ventilated as required by the sensor.

Block Diagram



]

Working Principle

Environmental signals can indicate changing conditions, but fire detection needs sensor fusion, robust thresholds, communications, and false-alarm handling. PIR detects changes in infrared patterns, not species identity.

Implementation Procedure

1.Test temperature/humidity readings indoors first.
2.Establish baseline values and document normal day/night variation.
3.Set conservative prototype alert thresholds and test with safe stimuli only.
4.Verify data logging and communication failure behavior.
5.For wildlife observation, use privacy-respecting placement and avoid disturbing animals.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// ESP32: DHT22 + PIR event monitor
#include <DHT.h>
#define DHTPIN 4
#define DHTTYPE DHT22
DHT dht(DHTPIN,DHTTYPE);
const int PIR=27, ALERT_LED=2;
const float TEMP_ALERT=40.0; // demonstration threshold only
void setup() {
  Serial.begin(115200); dht.begin();
  pinMode(PIR,INPUT); pinMode(ALERT_LED,OUTPUT);
}
void loop() {
  float t=dht.readTemperature(), h=dht.readHumidity();
  bool valid=!(isnan(t)||isnan(h));
  bool motion=digitalRead(PIR);
  bool alert=motion || (valid && t>=TEMP_ALERT);
  digitalWrite(ALERT_LED,alert?HIGH:LOW);
```

```
  if(valid) Serial.printf("temp=%.1fC humidity=%.1f%% ",t,h);
  else Serial.print("DHT_ERROR ");
  Serial.printf("motion=%d alert=%d\n",motion,alert);
  delay(2000);
}
```

Proposed Test Plan

Test condition	Expected observation
Normal indoor conditions	Values are logged without unexpected alert.
Safe warm-air stimulus	Temperature trend changes gradually.
Motion across PIR field	Event is registered.
Sensor/network loss	Failure is visibly indicated.

Expected Results

The node should report environmental readings and event triggers. It must not be described as proven wildfire prediction or reliable wildlife identification.

Limitations, Safety, and Future Scope

Outdoor weather, condensation, smoke composition, sensor drift, and false triggers affect performance. Never test fire or smoke alarms using uncontrolled flames or hazardous smoke.

Future scope: Add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.

EXP NO:7 Sensor-Based Failure Detection and Maintenance Scheduling
DATE:

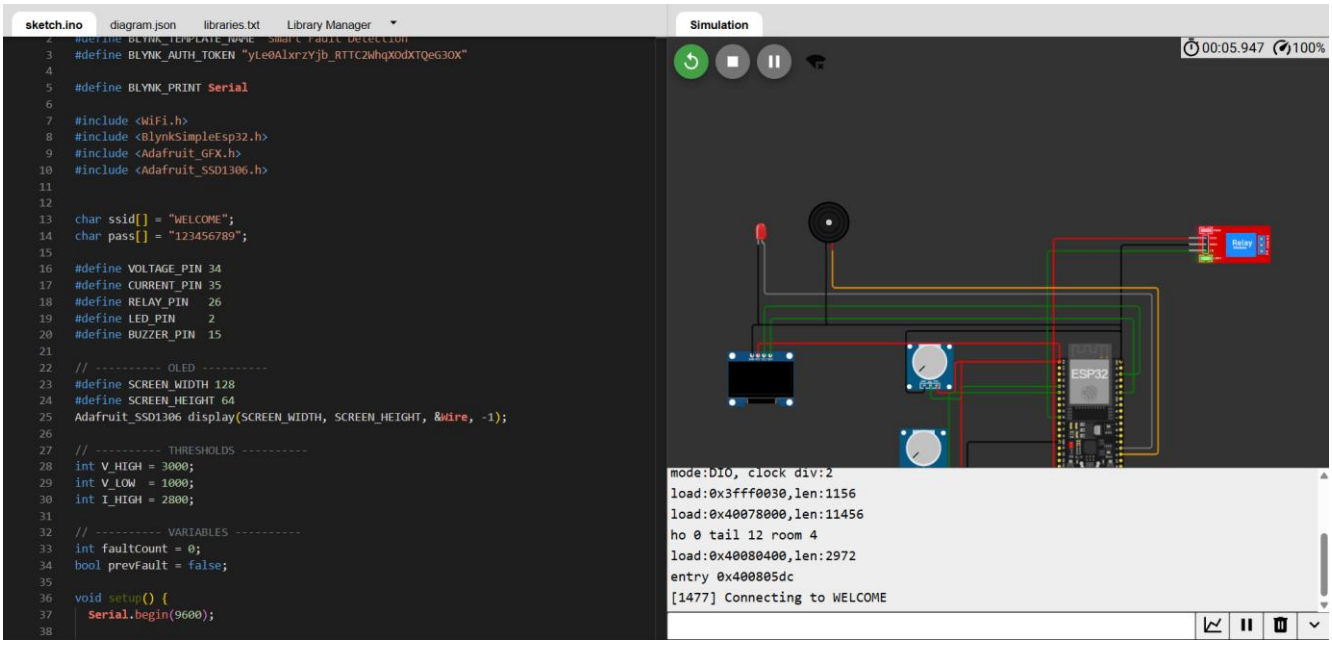


Figure 7. Sensor based failure detection and maintenance scheduling

Application domain: Condition-based maintenance

Abstract

A generic monitoring node logs selected sensor channels, counts out-of-range events, and helps demonstrate maintenance prioritization. The exact sensor set should be selected for the target asset.

Objectives

- Monitor one or more operational parameters.
- Record threshold excursions and uptime.
- Demonstrate a simple event counter and maintenance flag.
- Explain how historical data supports scheduling decisions.

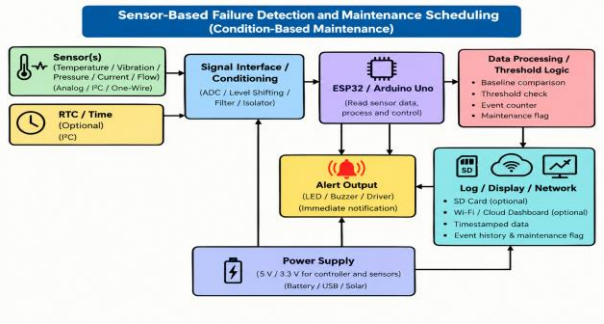
Components Required

ESP32; selected sensor (temperature, vibration, pressure, current, or flow); RTC or network time; SD card or dashboard optional; LED/buzzer.

Interface and Pin Connections

Use the selected sensor's interface: analog sensor to ADC, I²C sensor to SDA/SCL, or one-wire sensor to a digital pin. Check sensor voltage, isolation, and industrial signal conditioning before field installation.

Block Diagram



Working Principle

Condition-based maintenance uses deviations from baseline and event history to prioritize inspection. A threshold counter is a first step; scheduling requires asset criticality, failure modes, and historical failure data.

Implementation Procedure

1.Choose the asset and parameter relevant to a known failure mode.
2.Capture a baseline across normal operating states.
3.Set warning and critical thresholds from vendor limits or engineering review.
4.Log timestamp, value, event count, and sensor health.
5.Review trends and validate alerts with an inspection before changing maintenance intervals.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// Arduino Uno generic analog condition monitor
const int SENSOR=A0, LED=8;
const int WARN_LIMIT=700; // set after calibration
unsigned long alertCount=0;
bool wasAlert=false;
void setup() {
  Serial.begin(9600); pinMode(LED,OUTPUT);
}
void loop() {
  int value=analogRead(SENSOR);
  bool alert=value>WARN_LIMIT;
  if(alert && !wasAlert) alertCount++;
  wasAlert=alert;
  digitalWrite(LED,alert?HIGH:LOW);
  Serial.print("value="); Serial.print(value);
  Serial.print(", alert="); Serial.print(alert);
  Serial.print(", alert_count="); Serial.println(alertCount);
  delay(500);
}
```

Proposed Test Plan

Test condition	Expected observation
Nominal input	No warning is reported.
Input crosses warning threshold	Warning flag and event counter change.
Input returns to normal	Current warning clears according to defined logic.
Power cycle	Counters persist only if nonvolatile logging is implemented.

Expected Results

The system should classify readings against configured thresholds and log events. It does not establish remaining useful life without a validated model.

Limitations, Safety, and Future Scope

Thresholds must be asset-specific. Incorrect sensor mounting, scaling, or calibration can cause missed faults or false alarms.

Future scope: Add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.

EXP NO:8 Smart Retail Inventory and Customer-Experience Monitoring DATE:

Application domain: Retail automation

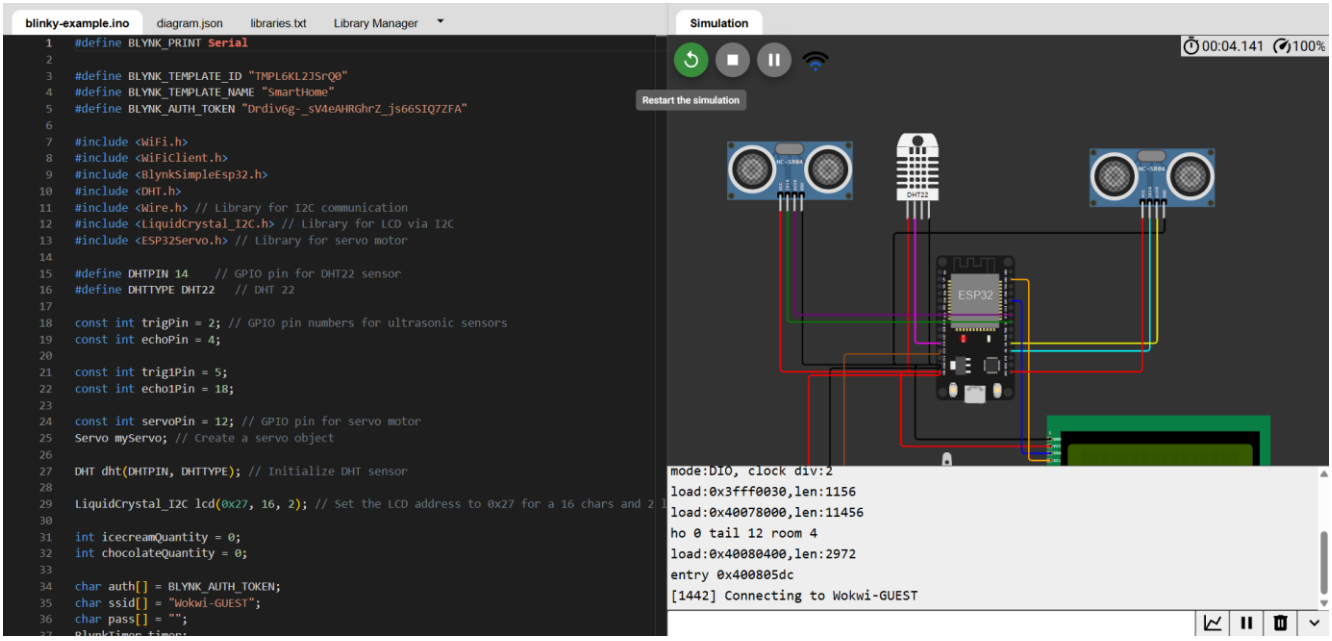


Figure 8 : Smart Retail Inventory and Customer-Experience Monitoring

Abstract

Demonstrates shelf-stock sensing and product-presence monitoring. It can estimate whether a designated shelf position is occupied and send a restock flag; customer personalization is outside the scope of this basic sensor node.

Objectives

- Detect product presence at a shelf location.
- Count removal/restock events where sensor behavior supports it.
- Generate a low-stock/restock notification.
- Discuss privacy and data-minimization requirements.

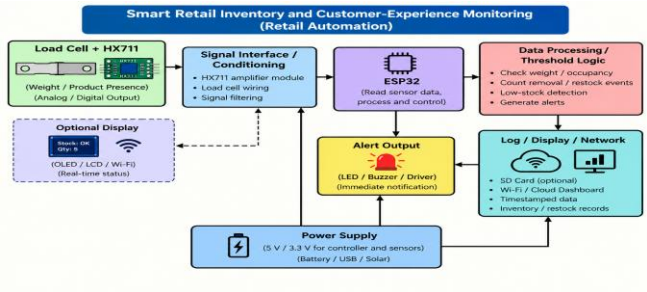
Components Required

ESP32; load cell with HX711 or IR break-beam; status LED; optional Wi-Fi dashboard; shelf fixture; stable power supply.

Interface and Pin Connections

HX711 DT/SCK → two digital pins (e.g. GPIO19/GPIO18 on ESP32); load cell wiring follows its amplifier module; LED → GPIO2 through resistor. Calibrate the load cell with known weights.

Block Diagram



Working Principle

A load cell measures force/weight through a strain-gauge bridge. A simple occupancy threshold can represent stock present/absent; multiple positions or a calibrated count model are needed for quantities.

Implementation Procedure

1.Mount the load cell mechanically so the shelf load transfers correctly.
2.Tare the empty platform and calibrate using a known weight.
3.Set occupied and low-stock thresholds based on product mass.
4.Test adding/removing items and compare sensor estimates with manual counts.
5.Avoid collecting identifiable customer data unless strictly necessary and consented.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// ESP32: HX711 load-cell threshold (install HX711 library)
#include "HX711.h"
HX711 scale;
const int DT_PIN=19, SCK_PIN=18, LED=2;
float EMPTY_LIMIT=20.0; // grams; calibrate scale first
void setup() {
  Serial.begin(115200);
  scale.begin(DT_PIN,SCK_PIN);
  scale.set_scale(1.0); // replace with calibration factor
  scale.tare();
  pinMode(LED,OUTPUT);
}
void loop() {
  if(scale.is_ready()) {
    float grams=scale.get_units(5);
    bool occupied=grams>EMPTY_LIMIT;
    digitalWrite(LED,occupied?HIGH:LOW);
    Serial.printf("weight=%.1fg occupied=%s\n",grams,occupied?"YES":"NO");
  } else Serial.println("HX711 not ready");
  delay(1000);
}
```

Proposed Test Plan

Test condition	Expected observation
Empty shelf position	Occupancy is classified as empty.
Known product placed	Measured weight crosses the occupied threshold.
Product removed	Restock state updates.
Varying product mass	Model limitations are documented.

Expected Results

A calibrated demonstration should detect weight changes and issue a restock state. Inventory accuracy depends on the number and consistency of items and mechanical setup.

Limitations, Safety, and Future Scope

Vibration, off-center loading, shelf friction, and different product weights affect readings. Weight sensing alone cannot reliably infer SKU identity or customer preferences.

Future scope: Add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.

EXP NO:9 Smart Logistics: Live Tracking and Shipment Security
DATE:

Application domain: Logistics and asset tracking

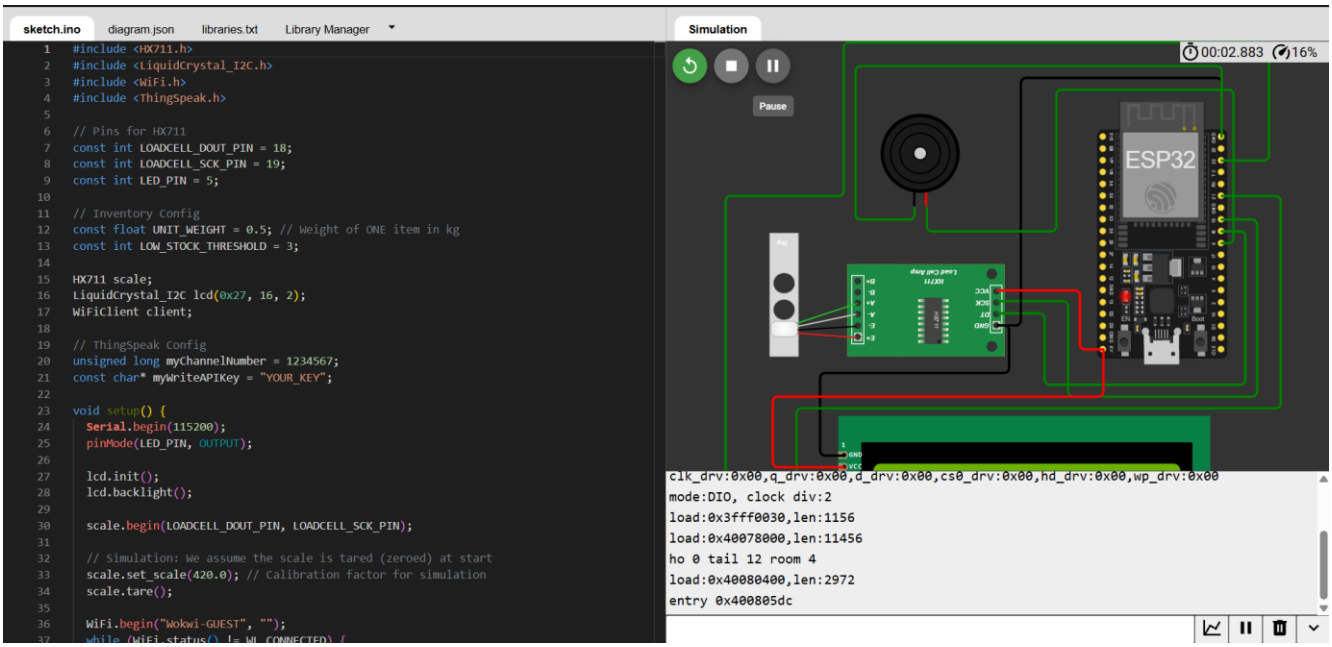


Figure 9 : Smart Logistics:Live tracking and shipment security

Abstract

A tracking node combines GNSS location with door/tamper or shock detection. It records the location and event status for a prototype shipment container.

Objectives

- Read location and time from a GNSS receiver.
- Detect a door/tamper event or shock event.
- Associate security events with location and timestamp.
- Explain coverage, battery, and data-protection limitations.

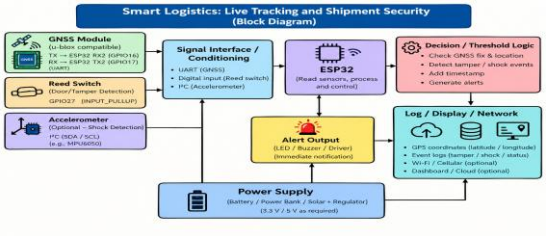
Components Required

ESP32; GNSS module (e.g. u-blox-compatible UART module); reed switch; optional accelerometer; battery/power regulator; optional cellular/Wi-Fi uplink.

Interface and Pin Connections

GNSS TX → ESP32 RX2 GPIO16 and GNSS RX → TX2 GPIO17 if needed; reed switch between GPIO27 and GND using INPUT_PULLUP. Confirm the GNSS module's logic levels and antenna view of sky.

Block Diagram



Working Principle

GNSS provides position estimates when satellite signals are available. A reed switch detects opening/closing; timestamps and coordinates can contextualize security events. Indoor or obstructed locations can prevent fixes.

Implementation Procedure

1.Verify UART output from the GNSS module in an open-sky location.
2.Read and validate latitude, longitude, fix quality, and timestamp.
3.Test the reed switch with the container open/closed.
4.Log an event only when the state changes; buffer events if the network is unavailable.
5.Secure credentials and restrict who can view tracking history.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// ESP32: reed-switch event + GNSS UART passthrough
const int REED=27;
void setup() {
  Serial.begin(115200);
  Serial2.begin(9600,SERIAL_8N1,16,17); // adjust to GNSS baud/pins
  pinMode(REED,INPUT_PULLUP);
}
void loop() {
  static int previous=HIGH;
  int state=digitalRead(REED);
  if(state!=previous) {
    Serial.printf("TAMPER_STATE=%s at_ms=%lu\n",
                  state==LOW?"CONTACT_CLOSED":"CONTACT_OPEN",millis());
    previous=state;
  }
  while(Serial2.available()) Serial.write(Serial2.read());
  delay(10);
}
```

Proposed Test Plan

Test condition	Expected observation
Open sky	GNSS may obtain a valid fix after startup.
Indoor test	No-fix condition is handled without reporting false coordinates.
Door opens	Tamper event is logged.
Network loss	Local sensing continues if designed for offline operation.

Expected Results

The prototype should report GNSS status when a fix is available and detect switch transitions. Continuous live tracking depends on a separate communications service and power budget.

Limitations, Safety, and Future Scope

GNSS can be inaccurate indoors and in urban canyons. Location histories are sensitive; provide access control and retention rules.

EXP NO:10 Connected Vehicle Performance and Remote Diagnostics
DATE:

Application domain: Automotive sensing

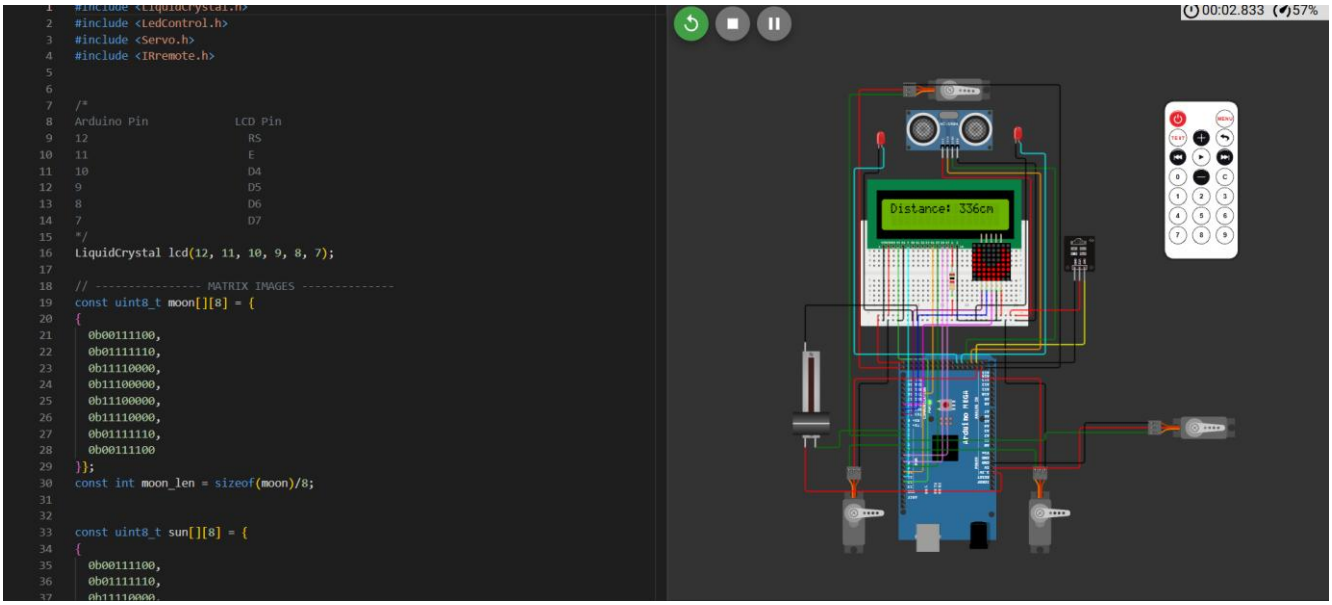


Figure 10 :Connected vehicle performance and remote diagnostics

Abstract

Uses non-invasive demonstration sensors to monitor temperature, vibration, or simulated engine data. Vehicle bus access and self-driving functions are intentionally excluded from this safe introductory prototype.

Objectives

- Acquire vehicle-related sensor data on a bench or simulator.
- Flag values outside a configured demonstration range.
- Log fault events with timestamps.
- Distinguish a classroom prototype from automotive-grade diagnostics.

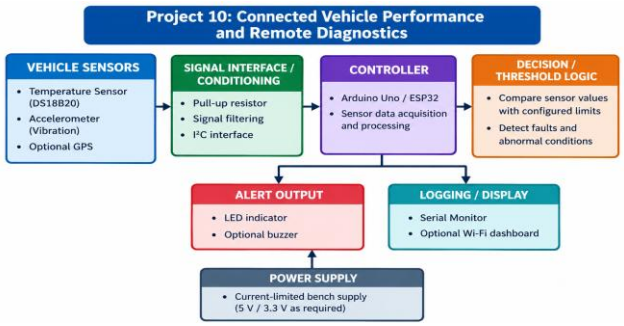
Components Required

ESP32 or Uno; DS18B20/temperature sensor; accelerometer; optional GPS; LED/buzzer; bench power supply. Do not connect directly to vehicle wiring or CAN bus without proper transceiver, isolation, and expertise.

Interface and Pin Connections

Temperature sensor to a digital pin with required pull-up; accelerometer via I²C or analog; alert LED through resistor. Use a current-limited bench supply for demonstration.

Block Diagram



Working Principle

Remote diagnostics combine sensor values, time, and fault rules. Production automotive diagnostics require qualified interfaces, electrical protection, validation, and compliance with vehicle safety requirements.

Implementation Procedure

1.Build and test on a bench rather than on a moving vehicle.
2.Use a known safe input range and validate the sensor's output.
3.Simulate fault conditions using controlled inputs.
4.Record each alert and its trigger condition.
5.If integrating with a real vehicle, use an approved diagnostic interface and professional guidance.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// Arduino Uno: temperature monitor with fault output
#include <OneWire.h>
#include <DallasTemperature.h>
OneWire oneWire(2);
DallasTemperature sensors(&oneWire);
const int ALERT_LED=8;
const float MAX_TEMP=80.0; // bench demonstration limit only
void setup() {
  Serial.begin(9600); sensors.begin();
  pinMode(ALERT_LED,OUTPUT);
}
void loop() {
  sensors.requestTemperatures();
  float t=sensors.getTempCByIndex(0);
  bool valid=(t>-100 && t<125);
  bool fault=(!valid || t>MAX_TEMP);
  digitalWrite(ALERT_LED,fault?HIGH:LOW);
  if(valid) Serial.print("temperature_C=");
  if(valid) Serial.println(t);
  else Serial.println("SENSOR_FAULT");
  delay(1000);
}
```

Proposed Test Plan

Test condition	Expected observation
Nominal bench input	No fault alert.
Temperature crosses threshold	Alert is raised.
Sensor disconnected	Fault or invalid reading is indicated.
Repeated event	Log records event transitions rather than every loop cycle.

Expected Results

The prototype should identify configured out-of-range inputs. It does not prove automotive diagnostic coverage or autonomous-driving capability.

Limitations, Safety, and Future Scope

Vehicle electrical transients, vibration, temperature, and safety requirements exceed ordinary breadboard design. Never use this prototype to control steering, braking, throttle, or other safety-critical systems.

Future scope: Add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.

EXP NO:11 Smart Building Facility Management and Access Control
DATE:

Application domain: Building automation

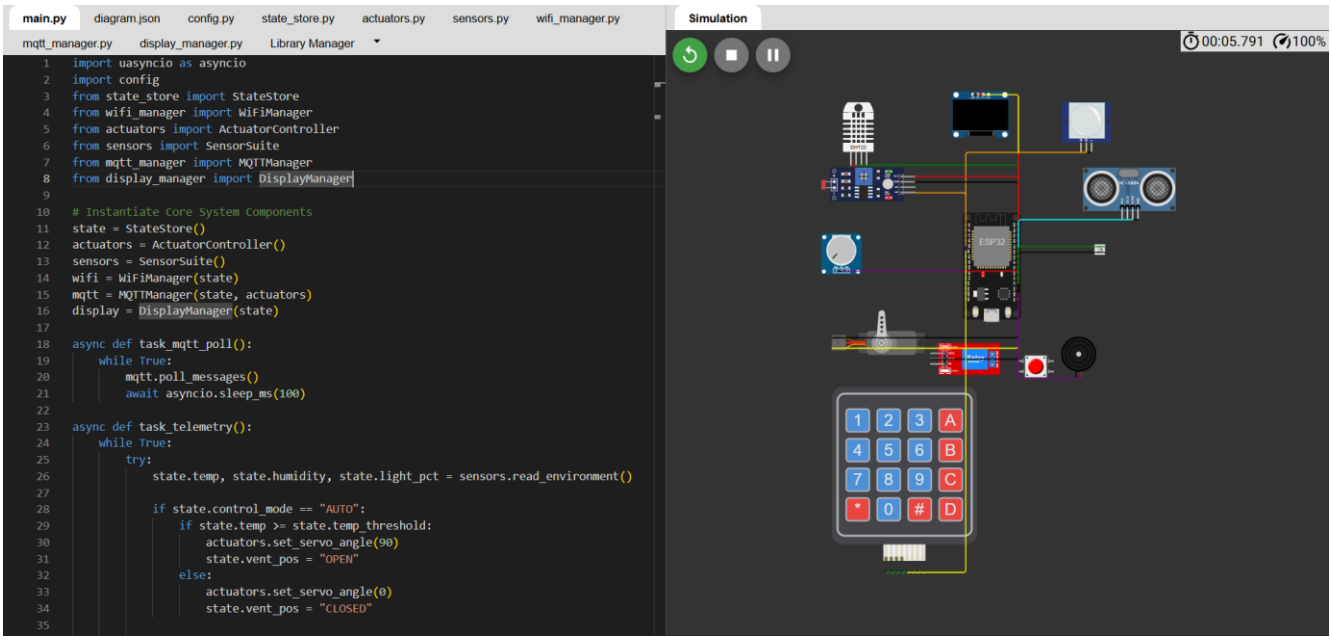


Figure 11:Smart building facility management and access control

Abstract

Demonstrates occupancy, environmental monitoring, and controlled access using an RFID reader or keypad. The prototype should control a low-voltage lock demonstrator, not a life-safety door.

Objectives

- Read occupancy and environmental conditions.
- Authenticate a demonstration RFID card or keypad entry.
- Log granted/denied events without storing unnecessary personal data.
- Show how energy control and access monitoring can be integrated.

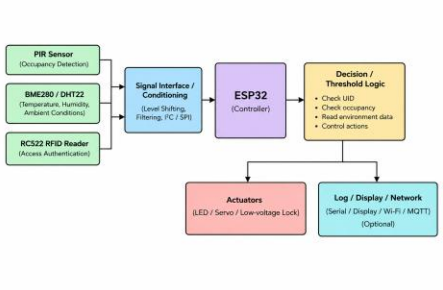
Components Required

ESP32; RC522 RFID module; PIR; BME280/DHT22; LED; servo or low-voltage lock demo; optional display.

Interface and Pin Connections

RC522 uses SPI and typically 3.3 V logic; follow the board-specific pinout. PIR OUT → GPIO27; I²C environmental sensor → GPIO21/22. Servo requires an appropriate external supply and common ground.

Block Diagram



Working Principle

RFID readers identify a tag's UID; UID-only authentication is not secure enough for high-security access. Occupancy and environmental readings can inform lighting or HVAC schedules.

Implementation Procedure

1.Wire the RFID module at its supported logic voltage.
2.Read a test tag UID and define a small demonstration allow-list.
3.Test access-denied behavior for unknown tags.
4.Connect only an LED/servo demonstration actuator.
5.For real access control, use secure credentials, audit logging, backup egress, and code-compliant hardware.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// ESP32 access-control logic template; connect RFID library separately
const int GRANT_LED=2, DENY_LED=4;
const String ALLOWED_UID="DE AD BE EF"; // replace with observed demo UID
void setup() {
  Serial.begin(115200);
  pinMode(GRANT_LED,OUTPUT); pinMode(DENY_LED,OUTPUT);
  digitalWrite(GRANT_LED,LOW); digitalWrite(DENY_LED,LOW);
  Serial.println("Read UID with your RC522 library, then call checkUID().");
}
void checkUID(String uid) {
  bool allowed=(uid==ALLOWED_UID);
  digitalWrite(GRANT_LED,allowed?HIGH:LOW);
  digitalWrite(DENY_LED,allowed?LOW:HIGH);
  Serial.println(allowed?"DEMO ACCESS GRANTED":"DEMO ACCESS DENIED");
}
void loop() {
  // Integrate MFRC522 library here and pass the scanned UID to checkUID().
  delay(100);
}
```

Proposed Test Plan

Test condition	Expected observation
Authorized demo tag	Grant indicator activates.
Unknown tag	Access is denied.
PIR event	Occupancy state changes.
Sensor or reader missing	System defaults to safe/locked state for the demo, without blocking emergency egress.

Expected Results

The prototype should distinguish preconfigured demo tags from unknown tags and report environmental data. It is not a security-certified access-control product.

Limitations, Safety, and Future Scope

RFID UID cloning and weak credential handling are possible. Never install hobby hardware on emergency exits or use it as the sole security barrier.

EXP NO:12 Smart Grid Monitoring and Renewable-Energy Integration
DATE:

Application domain: Energy monitoring

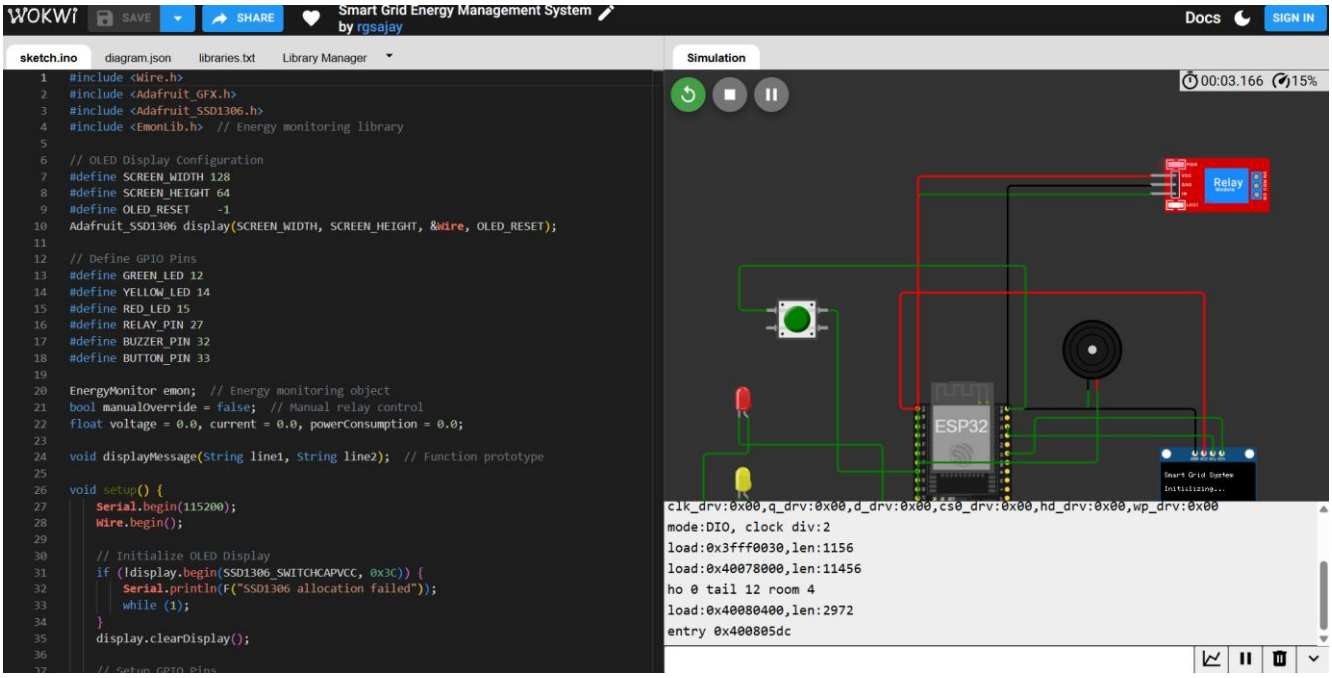


Figure 13:Smart grid monitoring and renewable energy integration

Abstract

Demonstrates low-voltage measurement of voltage/current and monitoring of a simulated solar or DC supply. Mains measurement must use certified isolated instruments and professional installation.

Objectives

- Measure low-voltage DC voltage through a designed divider or sensor module.
- Measure current with an appropriate current sensor.
- Calculate approximate power from voltage and current.
- Flag low-voltage or overcurrent conditions in a bench setup.

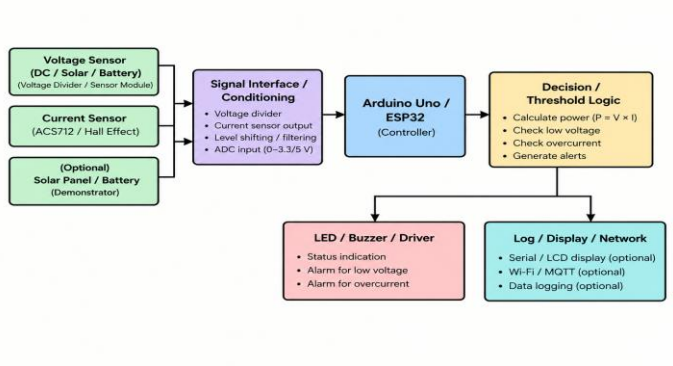
Components Required

ESP32 or Uno; low-voltage DC voltage sensor/divider; Hall-effect current sensor such as ACS712 module; LED; DC load; multimeter; optional solar panel/battery demonstrator.

Interface and Pin Connections

Voltage divider output → ADC, sized so the maximum stays within ADC input range; current sensor analog output → ADC; LED through resistor. ESP32 ADC maximum is board-dependent and typically 3.3 V; do not connect mains.

Block Diagram



Working Principle

For DC circuits, power is approximated by P = V × I. Grid-quality monitoring needs calibrated voltage/current waveforms, isolation, sampling, and standards-compliant equipment.

Implementation Procedure

1.Use a current-limited low-voltage DC supply only.
2.Verify divider ratio and sensor offset with a multimeter.
3.Measure voltage and current under several safe loads.
4.Compare computed power with a reference meter.
5.Document calibration and uncertainty.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// Arduino Uno: low-voltage DC voltage divider + analog current sensor
const int VOLT_PIN=A0, CURR_PIN=A1, LED=8;
const float ADC_REF=5.0, ADC_COUNTS=1023.0;
const float DIVIDER_RATIO=5.0; // replace with measured divider ratio
const float ACS_SENS=0.100; // example 100 mV/A; confirm exact module
float currentZeroV=2.50; // measure zero-current output
void setup() {
  Serial.begin(9600); pinMode(LED,OUTPUT);
}
void loop() {
  float vAdc=analogRead(VOLT_PIN)*ADC_REF/ADC_COUNTS;
  float voltage=vAdc*DIVIDER_RATIO;
  float iV=analogRead(CURR_PIN)*ADC_REF/ADC_COUNTS;
  float current=(iV-currentZeroV)/ACS_SENS;
  float power=voltage*current;
  digitalWrite(LED,(voltage<5.0 || fabs(current)>2.0)?HIGH:LOW);
  Serial.print("V="); Serial.print(voltage,2);
  Serial.print(" I="); Serial.print(current,2);
  Serial.print(" P_W="); Serial.println(power,2);
  delay(1000);
}
```

Proposed Test Plan

Test condition	Expected observation
No load	Current is near sensor offset/no-load value.
Known resistive load	Voltage and current change predictably.
Low supply voltage	Low-voltage flag activates if configured.
Reference comparison	Error is recorded rather than hidden.

Expected Results

The prototype should display approximate DC voltage, current, and power after calibration. It does not certify grid monitoring or protection performance.

Limitations, Safety, and Future Scope

Current sensor offset, ADC accuracy, heating, and wiring resistance affect readings. Never connect an Arduino/ESP32 directly to mains voltage.

Future scope: add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.

EXP NO:13 Precision Agriculture: Resource Use, Crop Health, and Yield Support

DATE:

Application domain: Precision agriculture

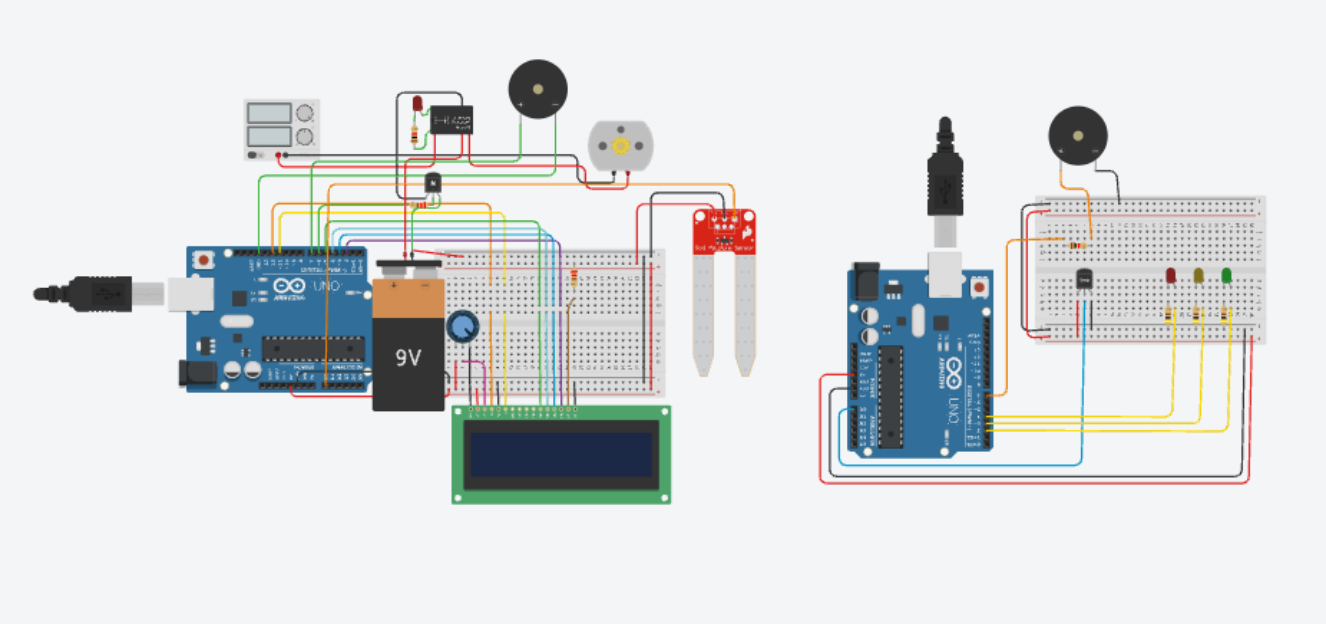


Figure 14:Precision agriculture :Resource use,crop health,and yield support

Abstract

Combines soil moisture, ambient temperature/humidity, and light sensing to create a basic crop-condition dashboard. Crop recommendations require crop-specific agronomy and calibration.

Objectives

- Collect soil moisture and environmental readings.
- Compare readings with configurable target bands.
- Flag dry soil or extreme environmental conditions.
- Support irrigation and field-observation planning.

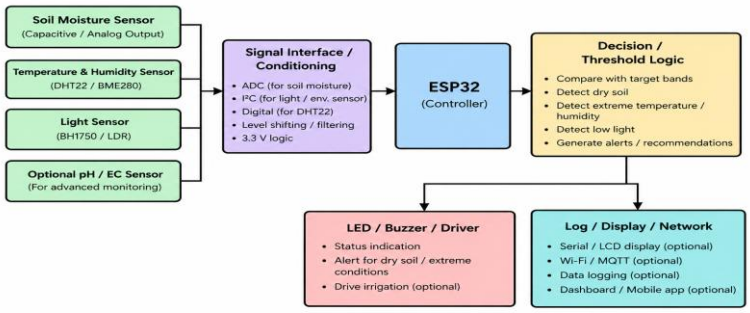
Components Required

ESP32; capacitive soil moisture sensor; DHT22/BME280; BH1750 or LDR; optional pH/EC sensors; solar/battery supply for field use.

Interface and Pin Connections

Soil moisture analog output to ADC; I²C light/environmental sensors to GPIO21/22; DHT22 data to GPIO4 if used. Ensure all module output voltages are safe for the controller.

Block Diagram



Working Principle

Crop monitoring combines root-zone water status and microclimate data. Moisture readings alone do not determine plant stress; soil texture, crop type, growth stage, and weather matter.

Implementation Procedure

1.Calibrate moisture sensor in representative soil.
2.Mount sensors at a meaningful root-zone depth and shield electronics from weather.
3.Log data at consistent intervals.
4.Compare readings with locally recommended crop-specific ranges.
5.Use readings as decision support, not automatic agronomic advice.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// ESP32: soil moisture + DHT22 crop-condition monitor
#include <DHT.h>
#define DHTPIN 4
#define DHTTYPE DHT22
DHT dht(DHTPIN,DHTTYPE);
const int SOIL_PIN=34;
const int DRY_RAW=3000; // calibrate on this soil
void setup() {
  Serial.begin(115200); dht.begin();
}
void loop() {
  int soil=analogRead(SOIL_PIN);
  float t=dht.readTemperature(), h=dht.readHumidity();
  bool dry=soil>DRY_RAW; // reverse if your sensor's behavior differs
  Serial.printf("soil_raw=%d soil_flag=%s ",soil,dry?"DRY":"OK");
  if(!isnan(t)&&!isnan(h)) Serial.printf("temp=%.1fC RH=%.1f%%\n",t,h);
  else Serial.println("DHT_ERROR");
  delay(3000);
}
```

Proposed Test Plan

Test condition	Expected observation
Dry vs moist soil	Moisture signal changes consistently.
Light changes	Light sensor reports a corresponding change.
Sensor failure	Missing data is flagged.
Daily log	Records are timestamped and units are explicit.

Expected Results

The system should collect multiple environmental variables and indicate values outside configured bands. Yield improvement must be evaluated in a controlled field trial.

Limitations, Safety, and Future Scope

Low-cost pH/EC probes need careful calibration and maintenance. Sensor placement and weather protection strongly affect data quality.

Future scope: Add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.

EXP NO:14 Smart Livestock Monitoring, Location Tracking, and Theft Detection

DATE:

Application domain: Livestock management

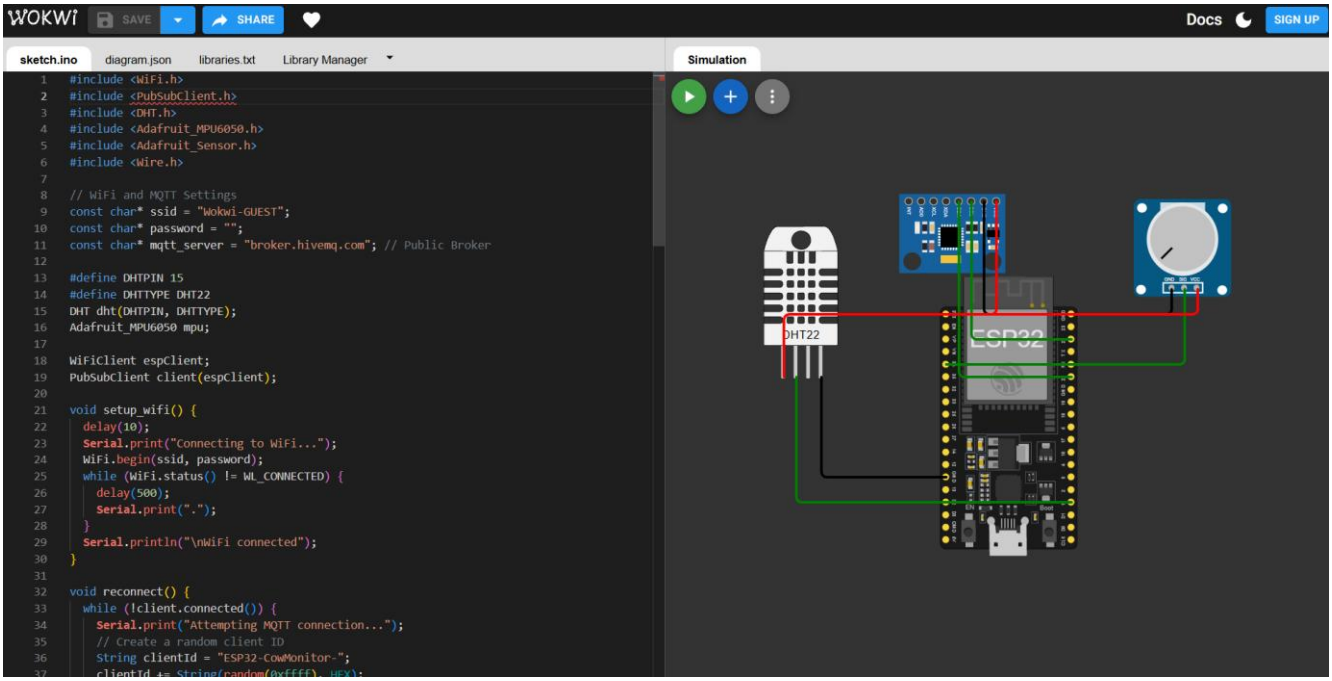


Figure 15:Smart livestock monitoring,location tracking and theft detection

Abstract

Demonstrates an animal-tag concept using location data and a simple enclosure/strap status input. Any wearable must be safe, lightweight, weather-resistant, and approved for the animal.

Objectives

- Read GNSS location when available.
- Detect a strap/tamper switch state or unusual motion event.
- Log location and event time.
- Discuss battery life, animal welfare, and geofence limitations.

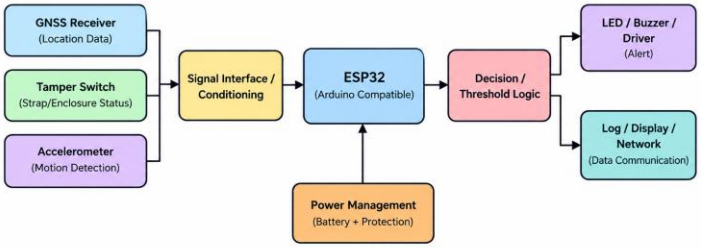
Components Required

ESP32; GNSS receiver; reed/limit switch or accelerometer; battery pack with protection; weather-resistant enclosure; optional LoRa/cellular radio.

Interface and Pin Connections

GNSS UART to ESP32 RX/TX; tamper switch between GPIO27 and GND using INPUT_PULLUP; accelerometer on I²C. Keep the enclosure secure and avoid sharp edges or heat buildup.

Block Diagram



Working Principle

GNSS supports outdoor location estimates; a switch detects enclosure or strap state. A geofence compares valid coordinates against a defined boundary and requires careful handling of poor fixes.

Implementation Procedure

1.Prototype on a bench before fitting to any animal.
2.Validate location fixes and timestamp handling outdoors.
3.Test tamper input with a switch, not by stressing an animal.
4.Estimate battery life from measured current draw and transmission interval.
5.Have animal-welfare and veterinary requirements reviewed before field deployment.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// ESP32: livestock-tag tamper input + GNSS serial output
const int TAMPER=27;
void setup() {
  Serial.begin(115200);
  Serial2.begin(9600,SERIAL_8N1,16,17);
  pinMode(TAMPER,INPUT_PULLUP);
}
void loop() {
  static int prev=HIGH;
  int s=digitalRead(TAMPER);
  if(s!=prev) {
    Serial.printf("tag_switch=%s time_ms=%lu\n",
                  s==LOW?"CLOSED":"OPEN",millis());
    prev=s;
  }
  while(Serial2.available()) Serial.write(Serial2.read());
  delay(10);
}
```

Proposed Test Plan

Test condition	Expected observation
Valid outdoor fix	Coordinates are shown with fix status.
No GNSS fix	No false location is reported.
Switch transition	Tamper event is logged.
Low battery simulation	Low-power state or warning is indicated if implemented.

Expected Results

The prototype should log valid location fixes and switch events. Theft prevention and animal-location accuracy are not established by bench tests.

Limitations, Safety, and Future Scope

GNSS may be unavailable under cover; wearable comfort and radio coverage are critical. Never use a prototype tag that can entangle, overheat, or injure an animal.

Future scope: Add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.

EXP NO:15 Remote Property Monitoring and Smart Security
DATE:

Application domain: Security sensing

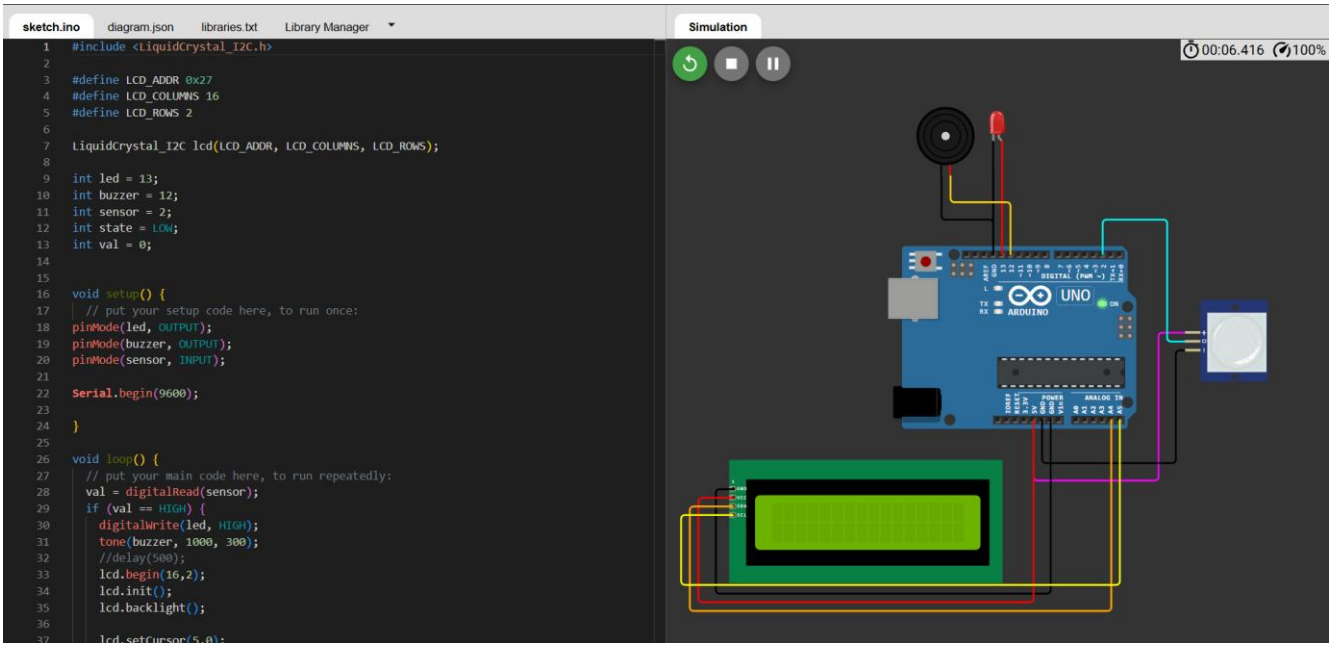


Figure 16:Remote property monitoring and smart security

Abstract

Uses door/window contacts, motion sensing, and optional light sensing to create a low-voltage intrusion-alert demonstrator.

Objectives

- Detect door/window state changes.
- Detect motion in a defined test area.
- Trigger a local indicator and record events.
- Provide an arm/disarm state and explain false-alarm reduction.

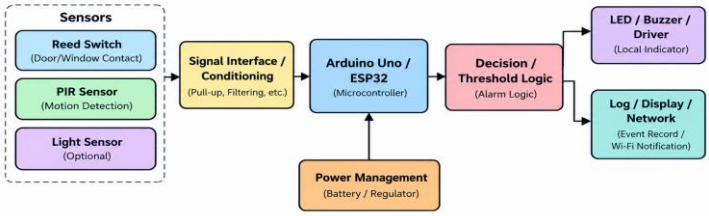
Components Required

ESP32 or Uno; magnetic reed switches; PIR sensor; buzzer through transistor driver; status LED; optional Wi-Fi notification.

Interface and Pin Connections

Reed switch between digital pin and GND using INPUT_PULLUP; PIR OUT to another digital pin; LED with series resistor; buzzer driver according to its current rating.

Block Diagram



Working Principle

A reed switch detects contact separation; PIR detects changing infrared patterns. Combining sensors and entry delay can reduce false alarms, but does not make the system a certified alarm.

Implementation Procedure

1.Test each sensor individually in a low-voltage setup.
2.Install contacts so door movement reliably changes switch state.
3.Add a visible armed/disarmed indicator.
4.Test false triggers from pets, heat sources, and airflow where appropriate.
5.Keep credentials secure and provide a safe way to silence the demonstrator.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// Arduino Uno: reed contact + PIR security demo
const int REED=2, PIR=3, LED=8, BUZZER=9;
bool armed=true;
void setup() {
  Serial.begin(9600);
  pinMode(REED,INPUT_PULLUP); pinMode(PIR,INPUT);
  pinMode(LED,OUTPUT); pinMode(BUZZER,OUTPUT);
}
void loop() {
  bool doorOpen=(digitalRead(REED)==HIGH);
  bool motion=(digitalRead(PIR)==HIGH);
  bool alarm=armed && (doorOpen || motion);
  digitalWrite(LED,alarm?HIGH:LOW);
  if(alarm) tone(BUZZER,1800); else noTone(BUZZER);
  Serial.printf("armed=%d door_open=%d motion=%d alarm=%d\n",
            armed,doorOpen,motion,alarm);
  delay(200);
}
```

Proposed Test Plan

Test condition	Expected observation
Closed contact	System reports closed.
Open contact while armed	Intrusion event is reported.
PIR motion	Motion event is reported.
Disarmed state	Alarm output remains disabled.

Expected Results

The system should register contact changes and motion and apply the selected arm/disarm logic. It does not guarantee prevention or detection of all intrusions.

Limitations, Safety, and Future Scope

Pets, thermal changes, switch misalignment, and power/network failure affect reliability. Do not rely on it as the sole security measure for valuable property.

Future scope: add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.

EXP NO:16 Smart Water Quality Monitoring
DATE:

Application domain: Water and environmental monitoring

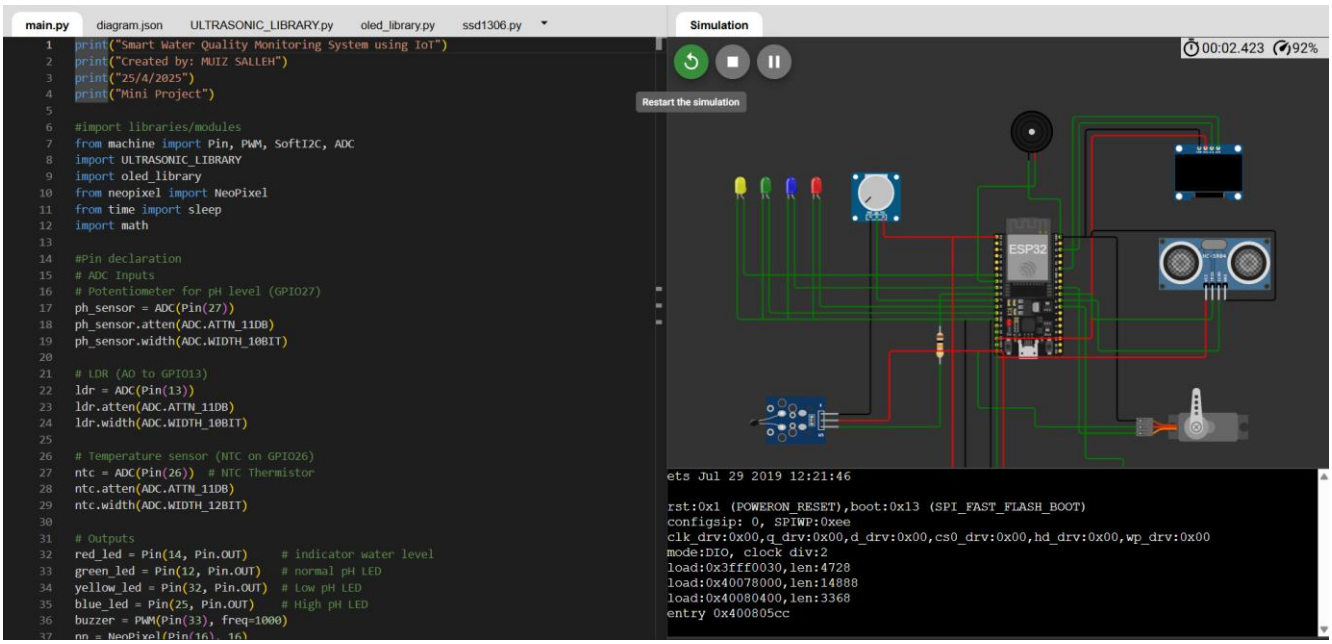


Figure 17 : Smart water quality monitoring

Abstract

Measures selected water-quality indicators such as temperature, pH, turbidity, or conductivity using appropriate probe modules. Sensor calibration and sample handling are central to meaningful results.

Objectives

- Acquire water temperature and one or more water-quality proxy measurements.
- Calibrate pH/turbidity/EC sensors according to manufacturer instructions.
- Flag readings outside user-defined demonstration bands.
- Explain that regulatory assessment requires validated methods.

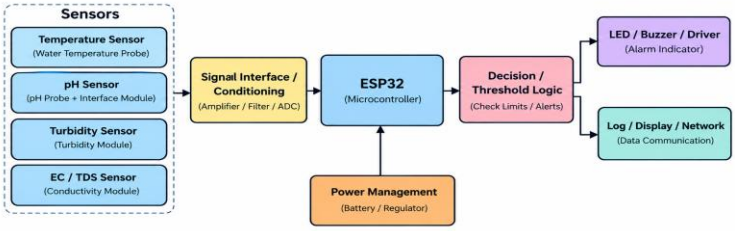
Components Required

ESP32; waterproof temperature probe; pH interface and probe; turbidity sensor; EC/TDS module; calibration solutions; sample containers; optional display.

Interface and Pin Connections

Temperature sensor to a digital pin; analog sensor modules to ADC pins through suitable conditioning; I²C display optional. Verify analog voltage range, especially for ESP32 3.3 V ADC.

Block Diagram



Working Principle

pH estimates acidity/alkalinity; turbidity estimates light scattering; conductivity relates to dissolved ions. These are different indicators and cannot by themselves certify drinking-water safety.

Implementation Procedure

1.Read the sensor datasheets and prepare recommended calibration solutions.
2.Calibrate each probe separately and rinse between samples.
3.Measure clean reference and test samples under consistent conditions.
4.Record temperature, calibration date, and sensor units.
5.Dispose of calibration solutions and samples according to their safety instructions.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// Arduino Uno: analog water sensor display template
const int PH_PIN=A0, TURBIDITY_PIN=A1;
const float VREF=5.0;
void setup() { Serial.begin(9600); }
void loop() {
  int phRaw=analogRead(PH_PIN);
  int turbRaw=analogRead(TURBIDITY_PIN);
  float phVoltage=phRaw*VREF/1023.0;
  float turbVoltage=turbRaw*VREF/1023.0;
  // Convert voltage to pH/turbidity only with your calibrated model.
  Serial.print("pH_interface_V="); Serial.print(phVoltage,3);
  Serial.print(", turbidity_interface_V="); Serial.println(turbVoltage,3);
  delay(1000);
}
```

Proposed Test Plan

Test condition	Expected observation
Calibration standard	Reading falls within the module's specified tolerance after calibration.
Two samples	Readings distinguish the samples when their properties differ.
Probe disconnected	Invalid measurement is reported.
Repeat sample	Repeatability is recorded.

Expected Results

Calibrated sensors should produce repeatable indicator readings within their specified limits. No claim of potability or regulatory compliance is made.

Limitations, Safety, and Future Scope

Cheap pH and turbidity modules drift and require maintenance. Water safety also depends on microbiological and chemical contaminants not measured by this setup.

Future scope: Add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.

EXP NO:17 Smart Parking Management and Payment-System Demonstrator

DATE:

Application domain: Smart mobility

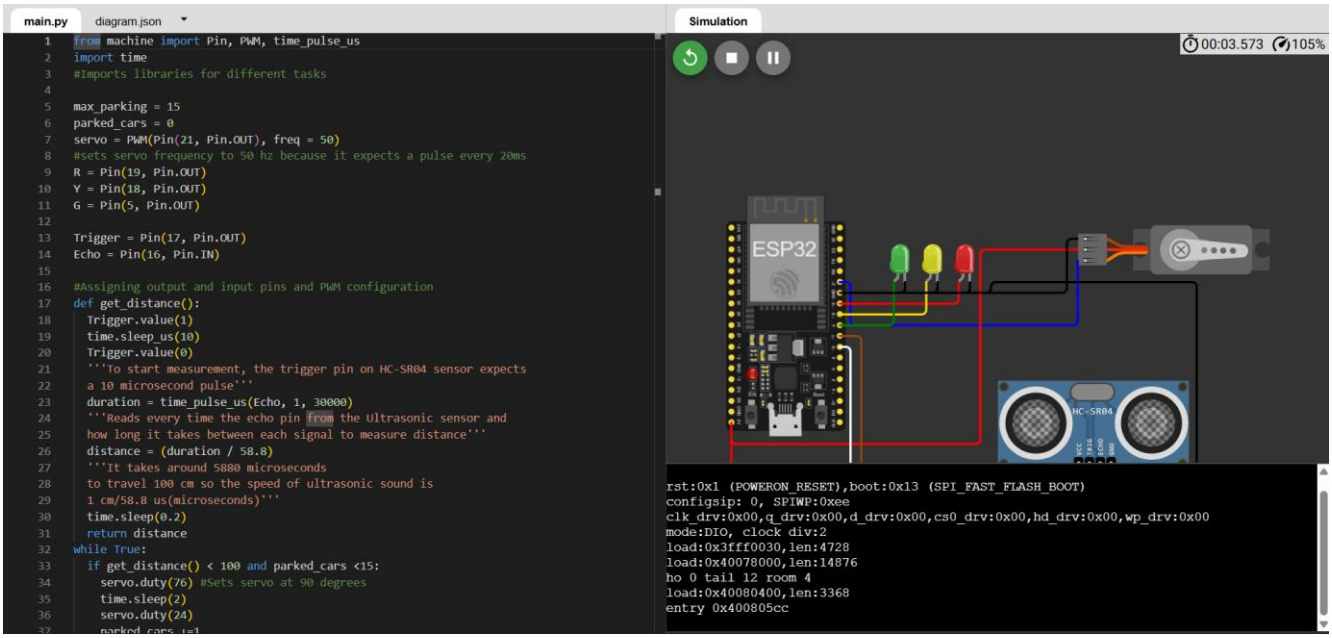


Figure 18 : Smart parking management and payment system demonstrator

Abstract

Detects occupancy for a small parking bay model and reports free/occupied status. Payment is represented as a mock status only; no real payment credentials are collected.

Objectives

- Detect whether a model parking space is occupied.
- Count available and occupied spaces.
- Display or transmit bay status.
- Demonstrate how payment status could be integrated without handling real financial data.

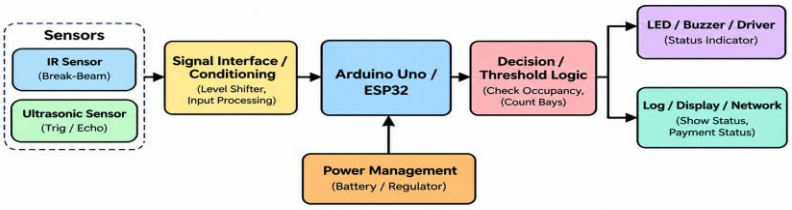
Components Required

ESP32; IR break-beam or ultrasonic sensor per bay; LEDs; OLED/LCD optional; model barrier servo optional.

Interface and Pin Connections

Digital IR output to GPIO input; ultrasonic TRIG/ECHO use a level shifter for ESP32 ECHO; LED pins use series resistors; servo needs an appropriate external supply and common ground.

Block Diagram



Working Principle

A sensor classifies each bay as occupied or vacant. A full system needs reliable detection, user interface, payment-provider integration, and fault handling.

Implementation Procedure

1.Calibrate sensor distance/beam for the model bay.
2.Test with different object sizes and positions.
3.Display occupancy and total free spaces.
4.Simulate payment approval using a local test flag only.
5.Test sensor failure and define a safe, clear status such as UNKNOWN.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// ESP32: one-bay ultrasonic occupancy demonstrator
const int TRIG=5, ECHO=18, LED=2;
const float OCCUPIED_CM=35.0; // calibrate for model
float readCm() {
  digitalWrite(TRIG,LOW); delayMicroseconds(2);
  digitalWrite(TRIG,HIGH); delayMicroseconds(10); digitalWrite(TRIG,LOW);
  long us=pulseIn(ECHO,HIGH,30000);
  return us ? us*0.0343/2.0 : -1;
}
void setup() {
  Serial.begin(115200);
  pinMode(TRIG,OUTPUT); pinMode(ECHO,INPUT); pinMode(LED,OUTPUT);
}
void loop() {
  float d=readCm();
  if(d<0) Serial.println("BAY=UNKNOWN");
  else {
    bool occupied=d<OCCUPIED_CM;
    digitalWrite(LED,occupied?HIGH:LOW);
    Serial.printf("distance=%.1fcm bay=%s\n",d,occupied?"OCCUPIED":"VACANT");
  }
  delay(500);
}
```

Proposed Test Plan

Test condition	Expected observation
Bay empty	Bay is marked vacant.
Model car present	Bay is marked occupied.
Object partly positioned	Misclassification is documented and threshold adjusted.
Sensor disconnected	Bay reports unknown/error if implemented.

Expected Results

The model should classify bay occupancy for the tested objects. Real-world detection accuracy and payment processing are outside the bench demonstration.

Limitations, Safety, and Future Scope

Rain, dirt, vehicle geometry, and sensor placement can cause errors. Use a legitimate payment provider for production and never store card details in the prototype.

Future scope: Add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.

EXP NO:18 Construction-Site Worker Safety and Operations Monitoring
DATE:

Application domain: Construction safety

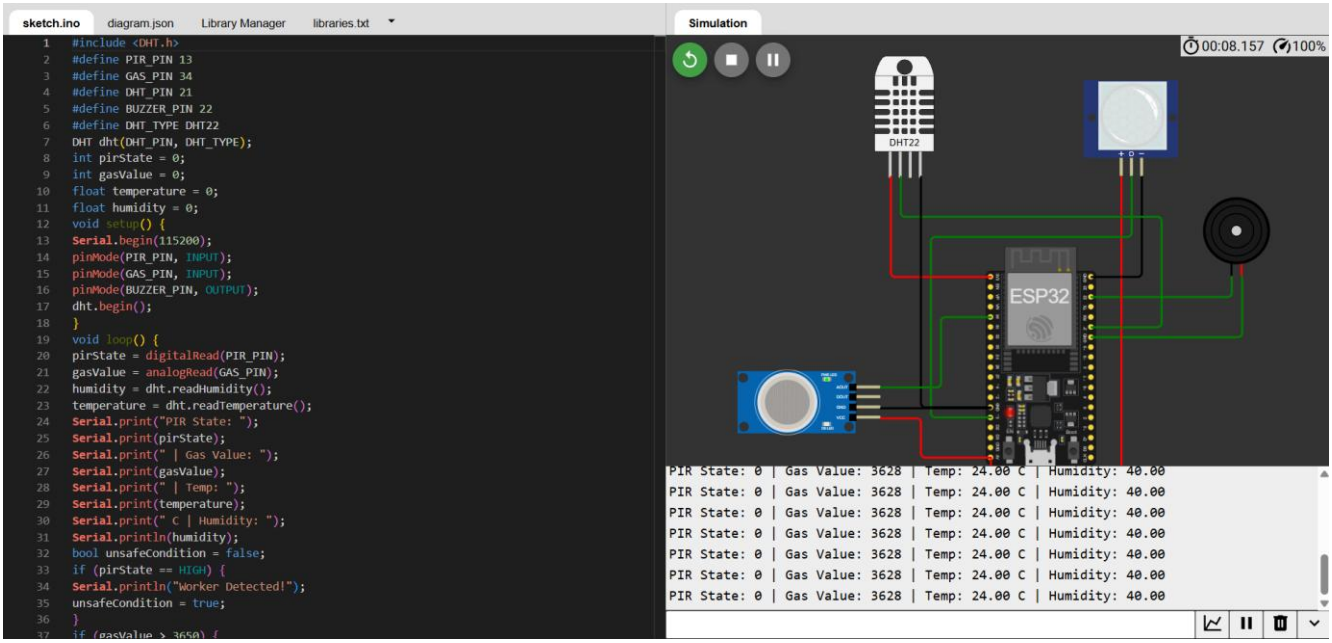


Figure 19 : Construction site worker safety and operations monitoring

Abstract

Demonstrates environmental alerts and motion/tilt sensing for a training model. It is not a certified personal protective equipment detector or a replacement for site safety procedures.

Objectives

- Monitor temperature or gas proxy in a controlled environment.
- Detect tilt or impact using an accelerometer.
- Trigger a local alert when a threshold is exceeded.
- Discuss worker privacy, calibration, and fail-safe procedures.

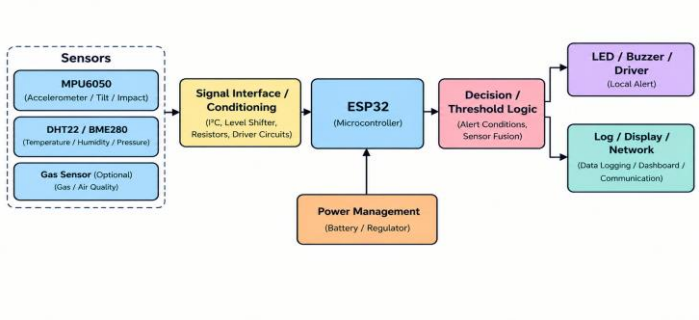
Components Required

ESP32; MPU6050 accelerometer; DHT22/BME280; optional approved gas sensor; buzzer/LED; enclosure; optional emergency-button input.

Interface and Pin Connections

MPU6050 and BME280 use I²C; alert LED through resistor; buzzer via transistor driver. Gas sensor modules may require heaters and higher current; follow datasheet requirements.

Block Diagram



Working Principle

Tilt/impact and environmental sensors can provide supplementary alerts. Reliable occupational safety requires hazard-specific certified equipment, site risk assessment, and human procedures.

Implementation Procedure

1.Mount sensors securely on a training fixture, not as an unapproved wearable.
2.Read baseline acceleration and environmental values.
3.Define and document test thresholds.
4.Use safe simulated events to trigger the alert.
5.Review false alarms and how the system reports sensor/network failure.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// ESP32: MPU6050 tilt-event template (Adafruit_MPU6050 library)
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
Adafruit_MPU6050 mpu;
const int ALERT_LED=2;
void setup() {
  Serial.begin(115200); pinMode(ALERT_LED,OUTPUT);
  if(!mpu.begin()) { Serial.println("MPU6050 not found"); while(true) delay(100); }
}
void loop() {
  sensors_event_t a,g,temp;
  mpu.getEvent(&a,&g,&temp);
  float tiltMagnitude=sqrt(a.acceleration.x*a.acceleration.x+
                           a.acceleration.y*a.acceleration.y);
  bool alert=tiltMagnitude>7.0; // demo only; calibrate safely
  digitalWrite(ALERT_LED,alert?HIGH:LOW);
  Serial.printf("ax=%.2f ay=%.2f az=%.2f tilt_proxy=%.2f alert=%d\n",
    a.acceleration.x,a.acceleration.y,a.acceleration.z,tiltMagnitude,alert);
  delay(200);
}
```

Proposed Test Plan

Test condition	Expected observation
Level fixture	Tilt estimate is near baseline.
Controlled tilt	Tilt event is detected.
Temperature threshold	Environmental alert activates.
Power/network interruption	Failure state is documented.

Expected Results

The prototype should detect controlled tilt and configured environmental thresholds. Worker injury prevention or compliance is not demonstrated.

Limitations, Safety, and Future Scope

Motion thresholds vary with mounting and activity. Do not use hobby sensors as the sole fall detector, gas alarm, or emergency safety system.

Future scope: add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.

EXP NO:19 Smart Waste Management and Sustainable Recycling
DATE:

Application domain: Waste management

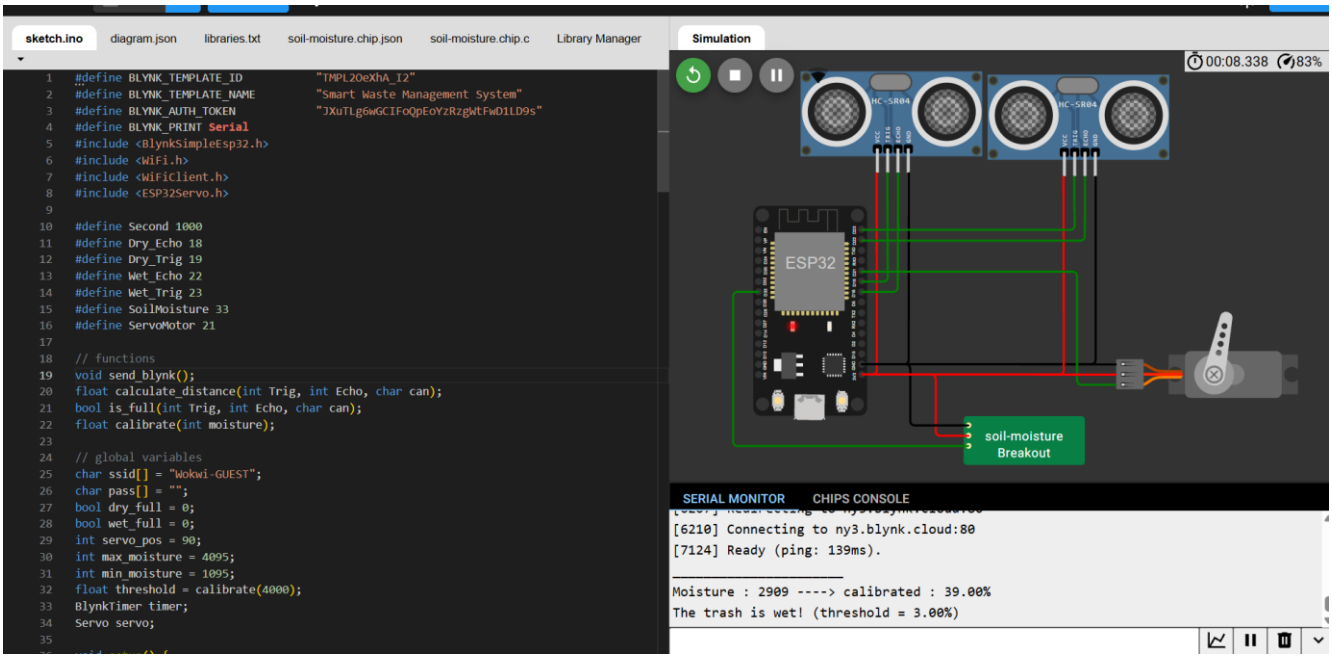


Figure 20 : Smart waste management and sustainable recycling

Abstract

Monitors bin fill level and optionally classifies material using a human-operated label or simple proximity input. It demonstrates route-planning data, not automatic material recognition.

Objectives

- Estimate bin fill level from distance.
- Generate a collection-needed flag.
- Record fill readings for collection planning.
- Discuss contamination, recycling rules, and sensor maintenance.

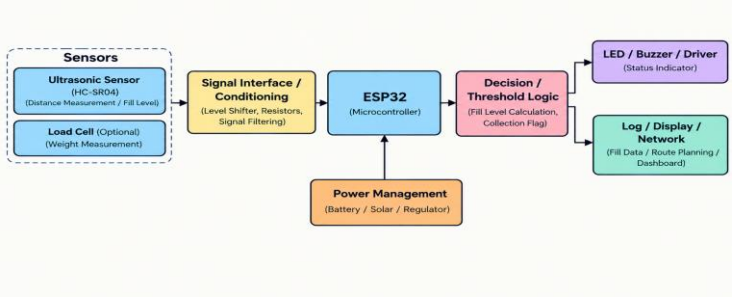
Components Required

ESP32; ultrasonic sensor; optional load cell; LED; weather-resistant bin-top enclosure; battery/solar supply optional.

Interface and Pin Connections

HC-SR04 TRIG → GPIO5; ECHO → GPIO18 through a level divider for ESP32; status LED → GPIO2 through resistor. Mount the sensor so the beam is not blocked by the lid.

Block Diagram



Working Principle

The sensor measures distance to the waste surface. Fill percentage can be estimated using known empty-bin depth, but irregular waste surfaces create uncertainty.

Implementation Procedure

1.Measure the usable bin depth and define empty/full distances.
2.Mount the sensor away from lid movement and condensation.
3.Collect readings with several safe test objects at different heights.
4.Set a collection threshold and add a sensor-error state.
5.Use fill data with collection routes and local recycling guidelines.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// ESP32: waste-bin fill estimate
const int TRIG=5, ECHO=18, LED=2;
const float EMPTY_DEPTH_CM=80.0;
const float FULL_THRESHOLD=80.0; // percent; adjust to collection policy
float getDistance() {
  digitalWrite(TRIG,LOW); delayMicroseconds(2);
  digitalWrite(TRIG,HIGH); delayMicroseconds(10); digitalWrite(TRIG,LOW);
  long us=pulseIn(ECHO,HIGH,30000);
  return us ? us*0.0343/2.0 : -1;
}
void setup() {
  Serial.begin(115200); pinMode(TRIG,OUTPUT);
  pinMode(ECHO,INPUT); pinMode(LED,OUTPUT);
}
void loop() {
  float d=getDistance();
  if(d<0 || d>EMPTY_DEPTH_CM+10) {
    Serial.println("SENSOR_ERROR");
    digitalWrite(LED,LOW);
  } else {
    float fill=constrain((EMPTY_DEPTH_CM-d)/EMPTY_DEPTH_CM*100.0,0,100);
    bool collect=fill>=FULL_THRESHOLD;
    digitalWrite(LED,collect?HIGH:LOW);
    Serial.printf("distance=%.1fcm fill_estimate=%.1f%% collect=%d\n",d,fill,collect);
  }
```

```
  delay(2000);
}
```

Proposed Test Plan

Test condition	Expected observation
Empty bin	Distance is near the empty reference.
Partly filled	Estimated fill increases.
Near-full model	Collection flag activates.
Soft/angled surface	Variation is noted and filtering adjusted.

Expected Results

The system should estimate fill level for tested surfaces and activate a collection flag at the selected threshold. Route efficiency and recycling improvement require field measurement.

Limitations, Safety, and Future Scope

Ultrasonic sensors may misread soft, angled, dusty, or wet surfaces. Fill percentage is an estimate and does not identify recyclable material.

Future scope: Add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.